

STATEMENT OF WORK – GOALKEEPER STATS FIFA 2026 – ANALYSIS TAKS 4



All the terrific matches of the FIFA World Cup 2026 have just ended. Advances in technology and football analytics leave a fascinating trove of team-level and player-level data wrangled from the FIFA official website.

FIFA Official site: [Worldcup - CanadaMexicoUSA 2026 - Statistics](#)

OBJECTIVE 1 - ANALYTIC QUESTION FORMULATION

Do goalkeepers with higher aggression (more actions outside the penalty area) have significantly different save efficiency compared to less aggressive goalkeepers? Nb. only goalkeeper that played in FIFA world cup 2026.

Thought's question - Are aggressive keepers:

- more efficient shot-stoppers
- or less efficient
- or no different at all

GITHUB: HIT140_Assessment-2_Group23

https://github.com/DanDooLoo/HIT140_Assessment-2_Group23

TASK 1 - DATA WRANGLING AND DATA PREPARATION

Data wrangling techniques

Many modern sports websites load data from a JSON API. This was how I was able to pull and create the data used in this assessment.

To find the data, I used the inspect function within the Microsoft Edge web browser from website: <https://www.fifa.com/en/tournaments/mens/worldcup/>

1. Open the FIFA stats page
- <https://www.fifa.com/en/tournaments/mens/worldcup/>
2. Press F12 on the keyboard
3. Opened the Network tab
4. Refreshed the page
5. Filtered by Fetch/XHR
6. Located for requests returning the following JSON:
https://play.fifa.com/json/dream_eleven/goalkeeper.json,
7. Opened the URL

The data then becomes a dictionary from the JSON-like data. Python-pandas code used:

```
import pandas as pd

goalkeeper = [
    {
        "id": 1798,
        "feedId": 512123,
        "fifaId": 482869,
        "firstName": "Jaouen",
        "lastName": "Hadjam",
        "knownName": "",
        "shortName": "Jaouen Hadjam",
        "squadId": 1,
        "position": "DEF",
        "status": "available"
    },
    {
        "id": 1799,
        "feedId": 126407,
        "fifaId": 379939,
        "firstName": "Nabil",
        "lastName": "Bentaleb",
        "knownName": "",
        "shortName": "Nabil Bentaleb",
        "squadId": 1,
        "position": "MID",
        "status": "available"
    }
]
```

```

    }
]

goalkeeper = pd.DataFrame(rows)

```

Data dictionary: [Appendix 1](#)

Goalkeeper csv information:

```

goalkeeper = player
- Rank
- Player
- Team
- Position
- Goalkeeper_Saves
- Goalkeeper_Actions_inside_PenaltyArea
- Goalkeeper_Actions_outside_PenaltyArea

```

DATA CLEANING

File: rounds.csv

```

import pandas as pd
import ast

# 1. Load CSV
df = pd.read_csv("rounds.csv")

# 2. Parse the JSON-like strings into Python objects
df["tournaments"] = df["tournaments"].apply(
    lambda x: ast.literal_eval(x) if isinstance(x, str) else None
)

# 3. Explode into match-level rows
matches = df.explode("tournaments").reset_index(drop=True)

# 4. Expand dict fields into columns
matches = pd.concat(
    [matches.drop(columns=["tournaments"]),
     matches["tournaments"].apply(pd.Series)],
    axis=1
)

# matches is now your clean match dataframe
print(matches.head())

# [5 rows x 25 columns] Columns:
# id
# venueName

```

```
# venueCity
# venueId
# date
# homeSquadId
# awaySquadId
# homeSquadName
# awaySquadName
# homeScore
# awayScore
# groupName
# matchDay
# plus the round metadata (stage, roundName, etc.)
```

DATA PREPARATION

Export created data to csv file

1. using python code: `goalkeeper.to_csv("goalkeeper.csv", index=False)`

Import csv into jupyter file

2. `import goalkeeper.csv`

Create backup file

3. created a backup of the goalkeeper data frames saving the data frame into notepad++
4. this is my master/backup if required

Import csv into jupyter notebook for analysis

5. `goalkeeper = pd.read_csv("goalkeeper.csv")`

Import required packages for analysis

6. pandas
7. numpy
8. scipy

ANALYSIS – GOALKEEPER AGGRESSION RATION – FIFA WORLD CUP 2026

Analysis: Step 1- Import goalkeeper.csv and create sample of dataset

1. Read goalkeeper.csv into Jupyter notebook, import required packages for analysis and add index.

```
# import required packages for stat analysis
import pandas as pd
import numpy as np
from scipy import stats as st
import statsmodels.stats.weightstats as stm
import math
import matplotlib.pyplot as plt
from scipy.stats import norm

# read in data collected from FIFA Official website
gk = pd.read_csv("goalkeeper.csv")
gk["Player_ID"] = gk.index + 1
```

```
gk.head()
```

	Rank	Player	Team	Position	Goalkeeper_Saves	Goalkeeper_Actions_inside_PenaltyArea	Goalkeeper_Actions_outside_PenaltyArea	Player_ID
0	1	Orlando Gill	PAR	GK	28	141	124	1
1	2	Eloy Room	CUW	GK	21	81	72	2
2	3	Emiliano Martinez	ARG	GK	20	111	97	3
3	3	Gregor Kobel	SUI	GK	20	100	88	4
4	3	Diogo Costa	POR	GK	20	114	93	5

```
gk_sample = gk.sample(30, random_state=42)
```

Figure 1 - Creating sample dataset

2. Create sample of the goalkeeper csv, selected 30 of the total 61 goalkkeepers.csv in the dataset:

```
gk_sample = gk.sample(30, random_state=42)
```

```
gk_sample.head()
```

	Rank	Player	Team	Position	Goalkeeper_Saves	Goalkeeper_Actions_inside_PenaltyArea	Goalkeeper_Actions_outside_PenaltyArea	Player_ID
69	58	Paul Izzo	AUS	GK	0	0	0	70
140	58	Geronimo Rulli	ARG	GK	0	0	0	141
27	28	Yahia Fofana	CIV	GK	11	84	68	28
19	20	Mostafa Shoubir	EGY	GK	14	119	105	20
42	43	Camilo Vargas	COL	GK	7	57	54	43

```
gk_sample.describe()
```

	Rank	Goalkeeper_Saves	Goalkeeper_Actions_inside_PenaltyArea	Goalkeeper_Actions_outside_PenaltyArea	Player_ID
count	30.000000	30.000000	30.000000	30.000000	30.000000
mean	46.533333	4.266667	27.433333	23.366667	74.600000
std	17.715634	6.274927	37.971571	32.571919	42.175822
min	9.000000	0.000000	0.000000	0.000000	10.000000
25%	37.750000	0.000000	0.000000	0.000000	38.500000
50%	58.000000	0.000000	0.000000	0.000000	75.500000
75%	58.000000	7.750000	57.000000	47.500000	111.250000
max	58.000000	17.000000	119.000000	105.000000	141.000000

Figure 2 - GK Sample header view

```
gk_sample.info()

<class 'pandas.DataFrame'>
Index: 30 entries, 69 to 9
Data columns (total 8 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Rank                                       30 non-null     int64
1   Player                                    30 non-null     str
2   Team                                       30 non-null     str
3   Position                                  30 non-null     str
4   Goalkeeper_Saves                         30 non-null     int64
5   Goalkeeper_Actions_inside_PenaltyArea    30 non-null     int64
6   Goalkeeper_Actions_outside_PenaltyArea    30 non-null     int64
7   Player_ID                                30 non-null     int64
dtypes: int64(5), str(3)
memory usage: 2.1 KB
```

Figure 3 - GK sample.info

Step 2 - Group goalkeepers into high and low aggression groups and save efficiency

1. Aggression Calculation

$$\text{Aggression Ratio} = \frac{\text{Outside Actions}}{\text{Inside Actions}}$$

2. Efficiency Calculation

$$\text{Save Efficiency} = \frac{\text{Goalkeeper Saves}}{\text{Total Actions}}$$

Group goalkeepers into two groups:

- > **High-aggression group** → above median aggression ratio
- > **Low-aggression group** → below median aggression ratio

Coding steps:

```
# Step 1:
# group goalkeepers into high and low aggression and save efficiency
# create variables
gk_sample["Aggression_Ratio"] = gk_sample["Goalkeeper_Actions_outside_PenaltyArea"] / gk_sample["Goalkeeper_Actions_inside_PenaltyArea"]
gk_sample["Save_Efficiency"] = gk_sample["Goalkeeper_Saves"] / (gk_sample["Goalkeeper_Actions_inside_PenaltyArea"] + gk_sample["Goalkeeper_Actions_outside_PenaltyArea"])

# Step 2.1:
# create medians to be able to sort goalkeepers into high and low group - aggression
median_aggression = gk_sample["Aggression_Ratio"].median()

# Step 2.2:
# create medians to be able to sort goalkeepers into high and low group - save
median_save = gk_sample["Save_Efficiency"].median()

# Step 3.1: --
# create high and low grouping of goalkeepers - aggression
high_aggression = gk_sample[gk_sample["Aggression_Ratio"] > median_aggression]
low_aggression = gk_sample[gk_sample["Aggression_Ratio"] <= median_aggression]

# Step 3.2: --
# create high and low grouping of goalkeepers - save
high_save = gk_sample[gk_sample["Save_Efficiency"] > median_save]
low_save = gk_sample[gk_sample["Save_Efficiency"] <= median_save]

# Review check point 1.1: - aggression
# check if it actually worked
print("High aggression group size: ", len(high_aggression))
print("Low aggression group size: ", len(low_aggression))

High aggression group size: 6
Low aggression group size: 6

# Review check point 1.2: - save
# check if it actually worked
print("High save group size: ", len(high_save))
print("Low save group size: ", len(low_save))

High save group size: 6
Low save group size: 6
```

Figure 4 - Creating GK groups

```
# Review check point 2.1: - aggression
# check if it actually worked - display header rows
high_aggression[["Player", "Team", "Aggression_Ratio"]].head()
```

	Player	Team	Aggression_Ratio
19	Mostafa Shoubir	EGY	0.882353
42	Camilo Vargas	COL	0.947368
18	Alisson	BRA	0.880597
11	Bart Verbruggen	NED	0.947368
51	Fernando Muslera	URU	0.829268

```
# Review check point 2.2: - save
# check if it actually worked - display header rows
high_save[["Player", "Team", "Save_Efficiency"]].head()
```

	Player	Team	Save_Efficiency
18	Alisson	BRA	0.119048
12	Thibaut Courtois	BEL	0.085000
36	Raul Rangel	MEX	0.077670
11	Bart Verbruggen	NED	0.153153
31	Matt Freese	USA	0.094340

```
# Review check point 3.1: - aggression
# check if it actually worked - display header rows
low_aggression[["Player", "Team", "Aggression_Ratio"]].head()
```

	Player	Team	Aggression_Ratio
27	Yahia Fofana	CIV	0.809524
12	Thibaut Courtois	BEL	0.818182
55	Mouhib Chamakh	TUN	0.769231
36	Raul Rangel	MEX	0.807018
31	Matt Freese	USA	0.827586

Figure 5 - Review created GK groups

```
In [140... # Review check point 3.2: - save
# check if it actually worked - display header rows
low_save[["Player", "Team", "Save_Efficiency"]].head()
```

```
Out[140...      Player Team Save_Efficiency
27    Yahia Fofana    CIV    0.072368
19  Mostafa Shoubir    EGY    0.062500
42    Camilo Vargas    COL    0.063063
55  Mouhib Chamakh    TUN    0.021739
45    Manuel Neuer    GER    0.066038
```

Figure 6 - Review cont.

```
high_aggression.describe()
```

	Rank	Goalkeeper_Saves	Goalkeeper_Actions_inside_PenaltyArea	Goalkeeper_Actions_outside_PenaltyArea	Player_ID	Aggression_Ratio	Save_Efficiency
count	6.000000	6.000000	6.000000	6.000000	6.000000	6.000000	6.000000
mean	24.666667	12.333333	71.166667	64.000000	26.000000	0.898989	0.092459
std	18.381150	5.501515	27.701384	24.503061	17.332051	0.045213	0.039519
min	9.000000	4.000000	41.000000	34.000000	10.000000	0.829268	0.053333
25%	10.500000	8.750000	57.000000	54.000000	13.750000	0.881036	0.062641
50%	17.500000	14.500000	62.000000	56.500000	19.500000	0.894665	0.083361
75%	37.250000	16.500000	81.250000	73.250000	37.250000	0.937271	0.115200
max	52.000000	17.000000	119.000000	105.000000	52.000000	0.947368	0.153153

```
low_aggression.describe()
```

	Rank	Goalkeeper_Saves	Goalkeeper_Actions_inside_PenaltyArea	Goalkeeper_Actions_outside_PenaltyArea	Player_ID	Aggression_Ratio	Save_Efficiency
count	6.000000	6.000000	6.000000	6.000000	6.000000	6.000000	6.000000
mean	34.000000	9.000000	66.000000	52.833333	35.333333	0.794874	0.069526
std	15.70987	5.25357	28.390139	23.752193	14.881756	0.034355	0.025398
min	9.000000	1.000000	26.000000	20.000000	13.000000	0.737705	0.021739
25%	29.000000	7.250000	57.250000	45.250000	29.000000	0.778677	0.067620
50%	34.000000	9.000000	59.500000	47.000000	34.500000	0.808271	0.075019
75%	41.250000	10.750000	78.250000	63.000000	43.750000	0.816017	0.083167
max	56.000000	17.000000	110.000000	90.000000	56.000000	0.827586	0.094340

Figure 7 - high aggression.describe

High Aggression - Described

Table 1 - High Aggression: Measures of Central Tendency

	Rank	GK Saves	Goalkeeper Actions Inside PenArea	Goalkeeper Actions Outside PenArea	Player ID	Aggression Ratio	Save Efficiency
count	6.000000	6.000000	6.000000	6.000000	6.000000	6.000000	6.000000
mean	24.666667	12.333333	71.166667	64.000000	26.000000	0.898989	0.092459
std	18.381150	5.501515	27.701384	24.503061	17.332051	0.045213	0.039519
min	9.000000	4.000000	41.000000	34.000000	10.000000	0.829268	0.053333
25%	10.500000	8.750000	57.000000	54.000000	13.750000	0.881036	0.062641
50%	17.500000	14.500000	62.000000	56.500000	19.500000	0.894665	0.083361
75%	37.250000	16.500000	81.250000	73.250000	37.250000	0.937271	0.115200
max	52.000000	17.000000	119.000000	105.000000	52.000000	0.947368	0.153153

Low Aggression - Described

Table 2 - Low Aggression: Measures of Central Tendency

	Rank	Goalkeeper Saves	Goalkeeper Actions Inside PenArea	Goalkeeper Actions Outside PenArea	Player ID	Aggression Ratio	Save Efficiency
count	6.00000	6.00000	6.000000	6.000000	6.000000	6.000000	6.000000
mean	34.00000	9.00000	66.000000	52.833333	35.333333	0.794874	0.069526
std	15.70987	5.25357	28.390139	23.752193	14.881756	0.034355	0.025398
min	9.00000	1.00000	26.000000	20.000000	13.000000	0.737705	0.021739
25%	29.00000	7.25000	57.250000	45.250000	29.000000	0.778677	0.067620
50%	34.00000	9.00000	59.500000	47.000000	34.500000	0.808271	0.075019
75%	41.25000	10.75000	78.250000	63.000000	43.750000	0.816017	0.083167
max	56.00000	17.00000	110.000000	90.000000	56.000000	0.827586	0.094340

high_save.describe()							
	Rank	Goalkeeper_Saves	Goalkeeper_Actions_inside_PenaltyArea	Goalkeeper_Actions_outside_PenaltyArea	Player_ID	Aggression_Ratio	Save_Efficiency
count	6.000000	6.00	6.000000	6.000000	6.000000	6.000000	6.000000
mean	18.333333	14.00	72.500000	62.500000	20.500000	0.864621	0.105478
std	12.420413	4.00	21.491859	17.683325	11.362218	0.056112	0.027485
min	9.000000	8.00	57.000000	46.000000	10.000000	0.807018	0.077670
25%	9.000000	11.25	57.250000	49.500000	12.250000	0.820533	0.087335
50%	12.000000	16.00	62.500000	56.500000	16.000000	0.854092	0.098999
75%	27.750000	17.00	81.250000	73.250000	28.750000	0.900382	0.115200
max	36.000000	17.00	110.000000	90.000000	37.000000	0.947368	0.153153
low_save.describe()							
	Rank	Goalkeeper_Saves	Goalkeeper_Actions_inside_PenaltyArea	Goalkeeper_Actions_outside_PenaltyArea	Player_ID	Aggression_Ratio	Save_Efficiency
count	6.000000	6.000000	6.000000	6.000000	6.000000	6.000000	6.000000
mean	40.333333	7.333333	64.666667	54.333333	40.833333	0.829242	0.056507
std	13.866026	4.676181	33.025243	29.776949	14.034481	0.076346	0.018112
min	20.000000	1.000000	26.000000	20.000000	20.000000	0.737705	0.021739
25%	31.750000	4.750000	45.000000	36.750000	31.750000	0.779304	0.055625
50%	43.000000	7.000000	59.000000	49.500000	44.500000	0.819396	0.062782
75%	49.750000	10.000000	78.250000	64.500000	50.500000	0.869082	0.065294
max	56.000000	14.000000	119.000000	105.000000	56.000000	0.947368	0.072368

Figure 8 - High save.describe

High Save Efficiency - Described

Table 3 - High Save Efficiency: Measures of Central Tendency

	Rank	GK Saves	GK Actions Inside PenArea	GK Actions Outside PenArea	Player ID	Aggression Ratio	Save Efficiency
count	6.000000	6.00	6.000000	6.000000	6.000000	6.000000	6.000000
mean	18.333333	14.00	72.500000	62.500000	20.500000	0.864621	0.105478
std	12.420413	4.00	21.491859	17.683325	11.362218	0.056112	0.027485
min	9.000000	8.00	57.000000	46.000000	10.000000	0.807018	0.077670
25%	9.000000	11.25	57.250000	49.500000	12.250000	0.820533	0.087335
50%	12.000000	16.00	62.500000	56.500000	16.000000	0.854092	0.098999
75%	27.750000	17.00	81.250000	73.250000	28.750000	0.900382	0.115200
max	36.000000	17.00	110.000000	90.000000	37.000000	0.947368	0.153153

Low Save Efficiency - Described

Table 4 - Low Save Efficiency: Measures of Central Tendency

	Rank	Goalkeeper Saves	Goalkeeper Actions Inside PenArea	Goalkeeper Actions Outside PenArea	Player ID	Aggression Ratio	Save Efficiency
count	6.000000	6.000000	6.000000	6.000000	6.000000	6.000000	6.000000
mean	40.333333	7.333333	64.666667	54.333333	40.833333	0.829242	0.056507
std	13.866026	4.676181	33.025243	29.776949	14.034481	0.076346	0.018112
min	20.000000	1.000000	26.000000	20.000000	20.000000	0.737705	0.021739
25%	31.750000	4.750000	45.000000	36.750000	31.750000	0.779304	0.055625
50%	43.000000	7.000000	59.000000	49.500000	44.500000	0.819396	0.062782
75%	49.750000	10.000000	78.250000	64.500000	50.500000	0.869082	0.065294
max	56.000000	14.000000	119.000000	105.000000	56.000000	0.947368	0.072368

LEVELS OF MEASUREMENT – MEASURES OF CENTRAL TENDENCY

Analysis: Step 2 - Create descriptive stats for high and low aggression groups

I used python to code central tendency of the file goalkeeper.csv.

Created – from the values in goalkeeper_saves, goalkeeper_inside_penaltyarea and goalkeeper_outside_penaltyarea:

- Mean
- Median
- Standard deviation
- Sample size

```
# Descriptive stats - mean, median, std dev for high aggression goalkeepers - Aggression ratio
high_aggression_mean = high_aggression["Aggression_Ratio"].mean()
high_aggression_median = high_aggression["Aggression_Ratio"].median()
high_aggression_std = high_aggression["Aggression_Ratio"].std()
high_aggression_n = len(high_aggression)
```

```
# Descriptive stats - mean, median, std dev for low aggression goalkeepers - Aggression ratio
low_aggression_mean = low_aggression["Aggression_Ratio"].mean()
low_aggression_median = low_aggression["Aggression_Ratio"].median()
low_aggression_std = low_aggression["Aggression_Ratio"].std()
low_aggression_n = len(low_aggression)
```

```
# Build a table (DataFrame) from calculated stats - Aggression Ratio
stats_table_aggression = pd.DataFrame({
    "Group": ["High Aggression", "Low Aggression"],
    "Mean Aggression Ratio": [high_aggression_mean, low_aggression_mean],
    "Median Aggression Ratio": [high_aggression_median, low_aggression_median],
    "Std Dev Aggression Ratio": [high_aggression_std, low_aggression_std],
    "N Goalkeepers": [high_aggression_n, low_aggression_n]
})

print(stats_table_aggression)
```

	Group	Mean Aggression Ratio	Median Aggression Ratio	\
0	High Aggression	0.898989	0.894665	
1	Low Aggression	0.794874	0.808271	

	Std Dev Aggression Ratio	N Goalkeepers
0	0.045213	6
1	0.034355	6

Figure 9 - GK coding Descriptive Statistics

```
# Descriptive stats - mean, median, std dev for high aggression goalkeepers - Save efficiency
high_save_mean = high_save["Save_Efficiency"].mean()
high_save_median = high_save["Save_Efficiency"].median()
high_save_std = high_save["Save_Efficiency"].std()
high_save_n = len(high_save)

# Descriptive stats - mean, median, std dev for low aggression goalkeepers - Save efficiency
low_save_mean = low_save["Save_Efficiency"].mean()
low_save_median = low_save["Save_Efficiency"].median()
low_save_std = low_save["Save_Efficiency"].std()
low_save_n = len(low_save)

# Build a table (DataFrame) from calculated stats - Save efficiency
stats_table_save = pd.DataFrame({
    "Group": ["High Save Efficiency", "Low Save Efficiency"],
    "Mean Save Efficiency": [high_save_mean, low_save_mean],
    "Median Save Efficiency": [high_save_median, low_save_median],
    "Std Dev Save Efficiency": [high_save_std, low_save_std],
    "N Goalkeepers": [high_save_n, low_save_n]
})

print(stats_table_save)
```

	Group	Mean Save Efficiency	Median Save Efficiency	\
0	High Save Efficiency	0.105478	0.098999	
1	Low Save Efficiency	0.056507	0.062782	

	Std Dev Save Efficiency	N Goalkeepers
0	0.027485	6
1	0.018112	6

Figure 10 - GK coding Descriptive Statistics

```
stats_table_save.style.format({
    "Mean Save Efficiency": "{:.3f}",
    "Median Save Efficiency": "{:.3f}",
    "Std Dev Save Efficiency": "{:.3f}"
})
```

	Group	Mean Save Efficiency	Median Save Efficiency	Std Dev Save Efficiency	N Goalkeepers
0	High Save Efficiency	0.105	0.099	0.027	6
1	Low Save Efficiency	0.057	0.063	0.018	6

Figure 11 - Create table of Descriptive Stats GK Save Efficiency

Table 5 - Goalkeepers Measures of Central Tendency – All goalkeepers

	GK Saves	GK Actions Inside PA	GK Actions Outside PA	Aggression Ratio	Save Efficiency
count	30	30	30	12	12
mean	4.2667	27.4333	23.3667	0.8469	0.0810
std	6.2749	37.9716	32.5719	0.0665	0.0339
min	0	0	0	0.7377	0.0217
25%	0	0	0	0.8089	0.0629
50%	0	0	0	0.8284	0.0750
75%	7.75	57	47.5	0.8885	0.0967
max	17	119	105	0.9474	0.1532

Table 6 - Goalkeepers Measures of Central Tendency – High and Low Aggression keepers – Save efficiency

	Group	Mean Save Efficiency	Median Save Efficiency	Std Dev Save Efficiency	N Goalkeepers
0	High Aggression	0.092	0.083	0.040	6
1	Low Aggression	0.070	0.075	0.025	6

INFERENCEAL STATISTICS - CONFIDENCE INTERVAL

The Central Limit Theorem - Confidence Intervals (CI) z scores

Confidence Level	Z Score
80%	1.282
85%	1.440
90%	1.645
95%	1.960
99%	2.576
99.5%	2.807
99.9%	3.291

Analysis: Step 3 - Compute confidence intervals (CI)

I will be using a 95% CI for the goalkeeper groups - high and low aggression as well as high and low save efficiency.

The general formula for a confidence interval around a mean is:

$$CI = \bar{X} \pm (Z \times (s / \sqrt{n}))$$

Where:

$\bar{X}$ - is the sample mean.

Z - is the Z-score.

s - is the sample standard deviation.

n - is the sample size.

Statistical calculation formula:

Equation 1 - Confidence Interval calculation

$$CI = \bar{X} \pm z^* \cdot \frac{s}{\sqrt{n}}$$

Calculating the confidence interval of goalkeepers – Defined Groups

Create visuals for aggression ratio

```
# calculation confidence interval goalkeeper high aggression
ci_low, ci_upp = stm._zconfint_generic(high_aggression_mean, high_aggression_n / math.sqrt(high_aggression_n), alpha=0.05, alternative='two-sided') # 95% confidence interval alpha=0.05 or 5%
print("CI high aggression goalkeepers: %.2f to %.2f" % (ci_low, ci_upp))

CI high aggression goalkeepers: -3.90 to 5.70

# calculation confidence interval goalkeeper low aggression
ci_low, ci_upp = stm._zconfint_generic(low_aggression_mean, low_aggression_n / math.sqrt(low_aggression_n), alpha=0.05, alternative='two-sided') # 95% confidence interval alpha=0.05 or 5%
print("CI low aggression goalkeepers: %.2f to %.2f" % (ci_low, ci_upp))

CI low aggression goalkeepers: -4.01 to 5.60
```

Figure 12 - Python coding - Create CI - GK Aggression Ratio

```
# High aggression CI
ci_low_high, ci_upp_high = stm._zconfint_generic(
    high_aggression_mean,
    high_aggression_n / math.sqrt(high_aggression_n),
    alpha=0.05,
    alternative='two-sided'
)

# Low aggression CI
ci_low_low, ci_upp_low = stm._zconfint_generic(
    low_aggression_mean,
    low_aggression_n / math.sqrt(low_aggression_n),
    alpha=0.05,
    alternative='two-sided'
)

# Build CI table
ci_table_aggression = pd.DataFrame({
    "Group": ["High Aggression", "Low Aggression"],
    "CI Lower": [ci_low_high, ci_low_low],
    "CI Upper": [ci_upp_high, ci_upp_low]
})

print(ci_table_aggression)
```

	Group	CI Lower	CI Upper
0	High Aggression	-3.901923	5.699900
1	Low Aggression	-4.006037	5.595786

Figure 13 - Aggression Ratio - Confidence Intervals

Create visuals for “Aggression_Ratio”

```
# CI midpoints (means)
high_aggression_mean_pltci = (ci_low_high + ci_upp_high) / 2
low_aggression_mean_plyci = (ci_low_high + ci_upp_high) / 2

# Galton bell curve (standard normal)
x = np.linspace(-10, 10, 500)
y = norm.pdf(x, 0, 1)

plt.figure(figsize=(12, 6))
plt.plot(x, y, label="Galton Bell Curve (Normal Distribution)", color="black")

# Plot CI ranges
plt.axvline(ci_low_high, color='red', linestyle='--', label='High Aggression CI Lower')
plt.axvline(ci_upp_high, color='red', linestyle='--', label='High Aggression CI Upper')
plt.axvline(high_aggression_mean_pltci, color='red', linestyle='-', label='High Aggression Mean')

plt.axvline(ci_low_high, color='blue', linestyle='--', label='Low Aggression CI Lower')
plt.axvline(ci_upp_high, color='blue', linestyle='--', label='Low Aggression CI Upper')
plt.axvline(low_aggression_mean_plyci, color='blue', linestyle='-', label='Low Aggression Mean')

# Add Labels
plt.title("Confidence Intervals on Galton's Bell Curve")
plt.xlabel("Aggression Score")
plt.ylabel("Probability Density")
plt.legend()
plt.grid(True)

plt.show()
```

Figure 14 - Python coding - Creating Galton's Curve (Distribution) - GK Aggression Ratio

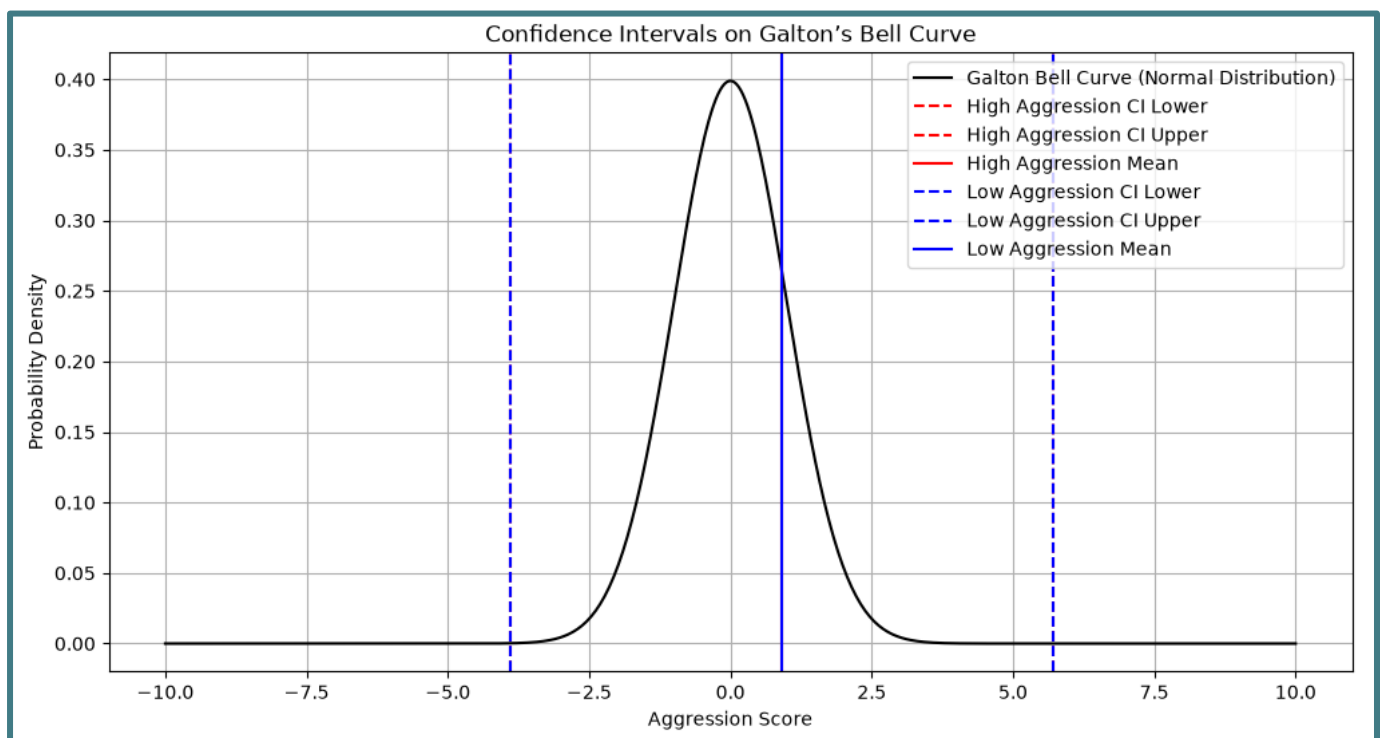


Figure 15 - Confidence Interval: Goalkeeper Aggression Ratio

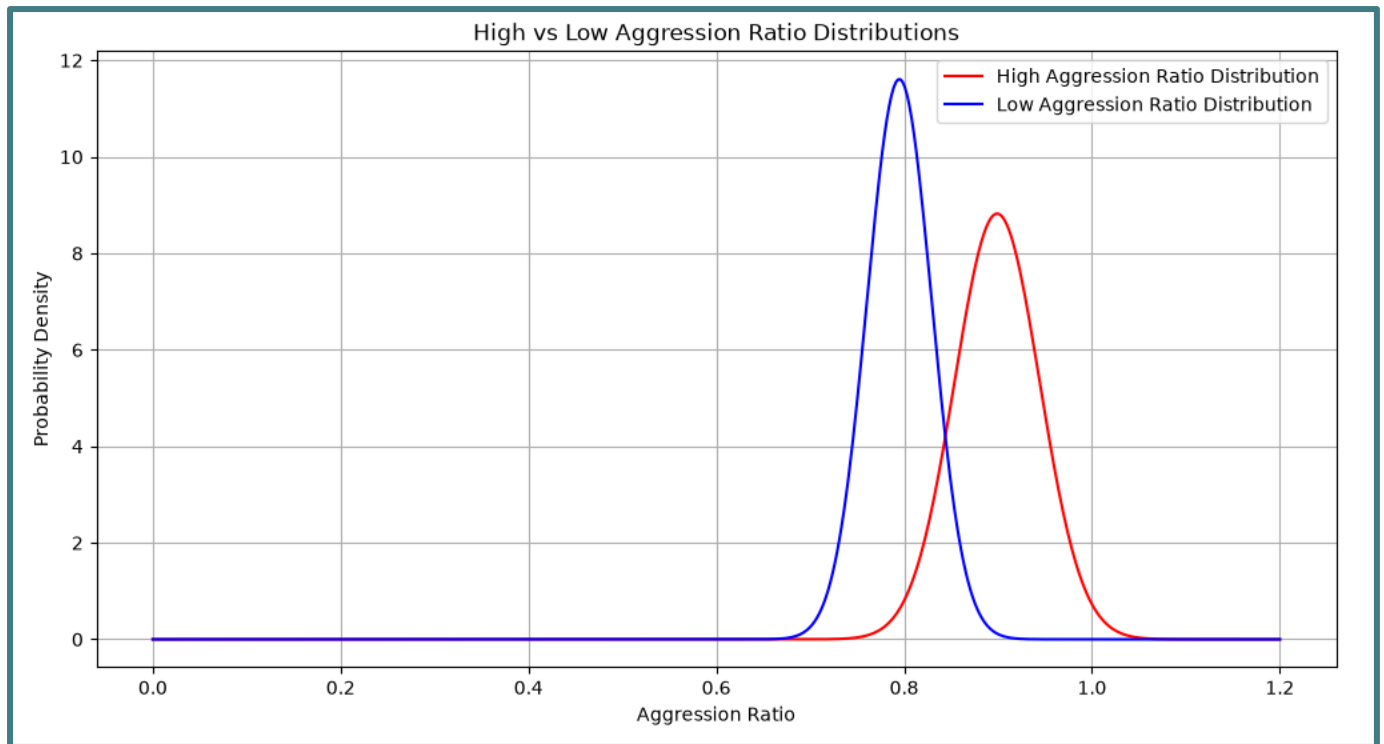


Figure 16 - High vs Low Aggression Distributions plot

Create visuals for “Save_Efficiency”

Create visuals for save efficiency

```
# calculation confidence interval goalkeeper save efficiency
ci_low, ci_upper = stm._zconfint_generic(high_save_mean, high_save_n / math.sqrt(high_save_n), alpha=0.05, alternative='two-sided') # 95% confidence interval alpha=0.05 or 5%
print("CI high save efficiency goalkeepers: %.2f to %.2f" % (ci_low, ci_upper))

CI high save efficiency goalkeepers: -4.70 to 4.91

# calculation confidence interval goalkeeper save efficiency
ci_low, ci_upper = stm._zconfint_generic(low_save_mean, low_save_n / math.sqrt(low_save_n), alpha=0.05, alternative='two-sided') # 95% confidence interval alpha=0.05 or 5%
print("CI low save efficiency goalkeepers: %.2f to %.2f" % (ci_low, ci_upper))

CI low save efficiency goalkeepers: -4.74 to 4.86
```

Figure 17 - Confidence Interval coding - Save Efficiency

```

# High save CI
ci_low_high_s, ci_upp_high_s = stm._zconfint_generic(
    high_save_mean,
    high_save_n / math.sqrt(high_save_n),
    alpha=0.05,
    alternative='two-sided'
)

# Low save CI
ci_low_low_s, ci_upp_low_s = stm._zconfint_generic(
    low_save_mean,
    low_save_n / math.sqrt(low_save_n),
    alpha=0.05,
    alternative='two-sided'
)

# Build CI table
ci_table_save = pd.DataFrame({
    "Group": ["High Save", "Low Save"],
    "CI Lower": [ci_low_high_s, ci_low_low_s],
    "CI Upper": [ci_upp_high_s, ci_upp_low_s]
})

print(ci_table_save)

```

	Group	CI Lower	CI Upper
0	High Save	-4.695434	4.906390
1	Low Save	-4.744405	4.857419

Figure 18 - Confidence Intervals: Save Efficiency

```

# CI midpoints (means) - save efficiency
high_save_mean_pltci = (ci_low_high_s + ci_upp_high_s) / 2
low_save_mean_plyci = (ci_low_high_s + ci_upp_high_s) / 2

# Galton bell curve (standard normal)
x = np.linspace(-10, 10, 500)
y = norm.pdf(x, 0, 1)

plt.figure(figsize=(12, 6))
plt.plot(x, y, label="Galton Bell Curve (Normal Distribution)", color="black")

# Plot CI ranges
plt.axvline(ci_low_high_s, color='red', linestyle='--', label='High Save CI Lower')
plt.axvline(ci_upp_high_s, color='red', linestyle='--', label='High Save CI Upper')
plt.axvline(high_save_mean_pltci, color='red', linestyle='-', label='High Save Mean')

plt.axvline(ci_low_high_s, color='blue', linestyle='--', label='Low Save CI Lower')
plt.axvline(ci_upp_high_s, color='blue', linestyle='--', label='Low Save CI Upper')
plt.axvline(low_save_mean_plyci, color='blue', linestyle='-', label='Low Save Mean')

# Add Labels
plt.title("Confidence Intervals on Galton's Bell Curve")
plt.xlabel("Save Score")
plt.ylabel("Probability Density")
plt.legend()
plt.grid(True)

plt.show()

```

Figure 19 - Plotting Galton's Curve: Save Efficiency

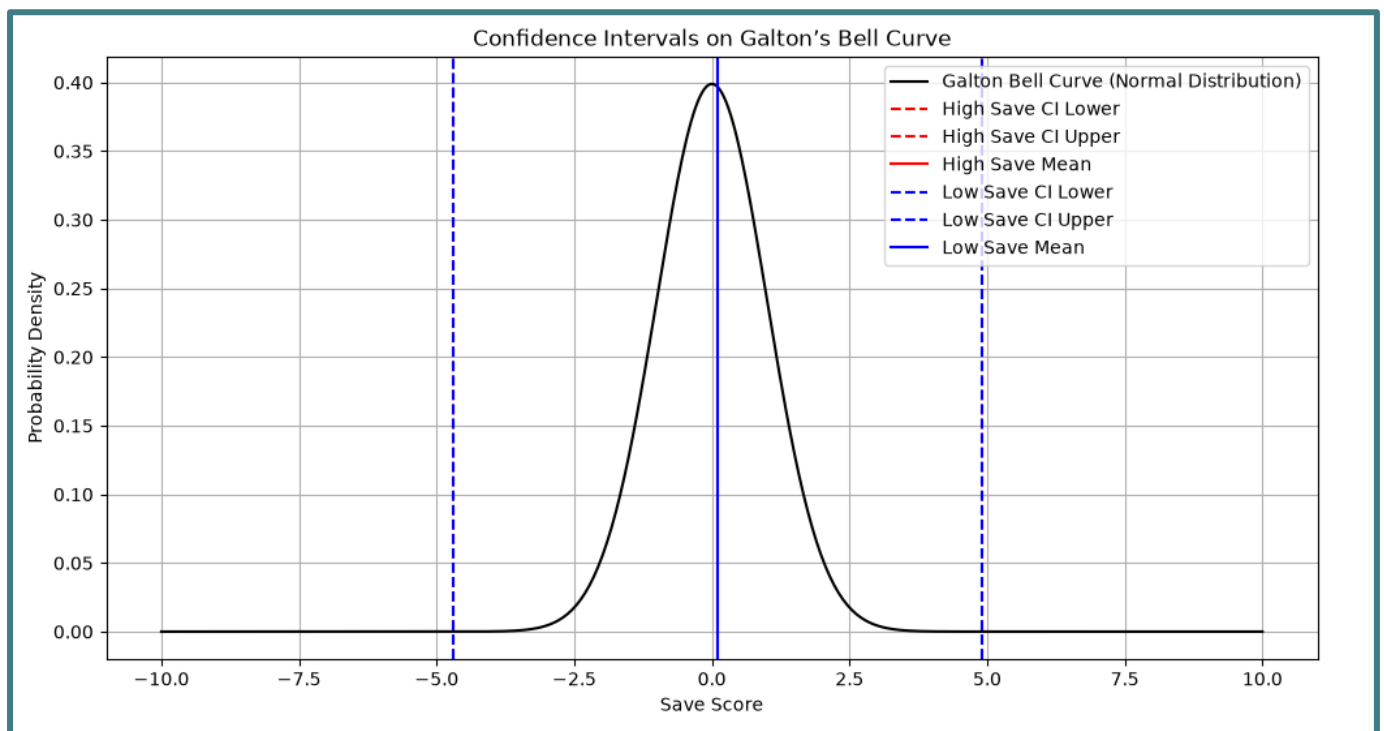


Figure 20 - Galton's Curve - Save Efficiency

```

high_save_sd_plt_c = high_save_std
low_save_sd_plt_c = low_save_std

# X range
x = np.linspace(0, 1.2, 500)

# PDFs
s_high_pdf = norm.pdf(x, high_save_mean, high_save_sd_plt_c)
s_low_pdf = norm.pdf(x, low_save_mean, low_save_sd_plt_c)

plt.figure(figsize=(12,6))

# Plot curves
plt.plot(x, s_high_pdf, color='red', label='High Save Efficiency Distribution')
plt.plot(x, s_low_pdf, color='blue', label='Low Save Efficiency Distribution')

# Labels
plt.title("High vs Low Save Efficiency Distributions")
plt.xlabel("Save Efficiency")
plt.ylabel("Probability Density")
plt.legend()
plt.grid(True)

plt.show()

```

Figure 21 - Python coding - Save Efficiency

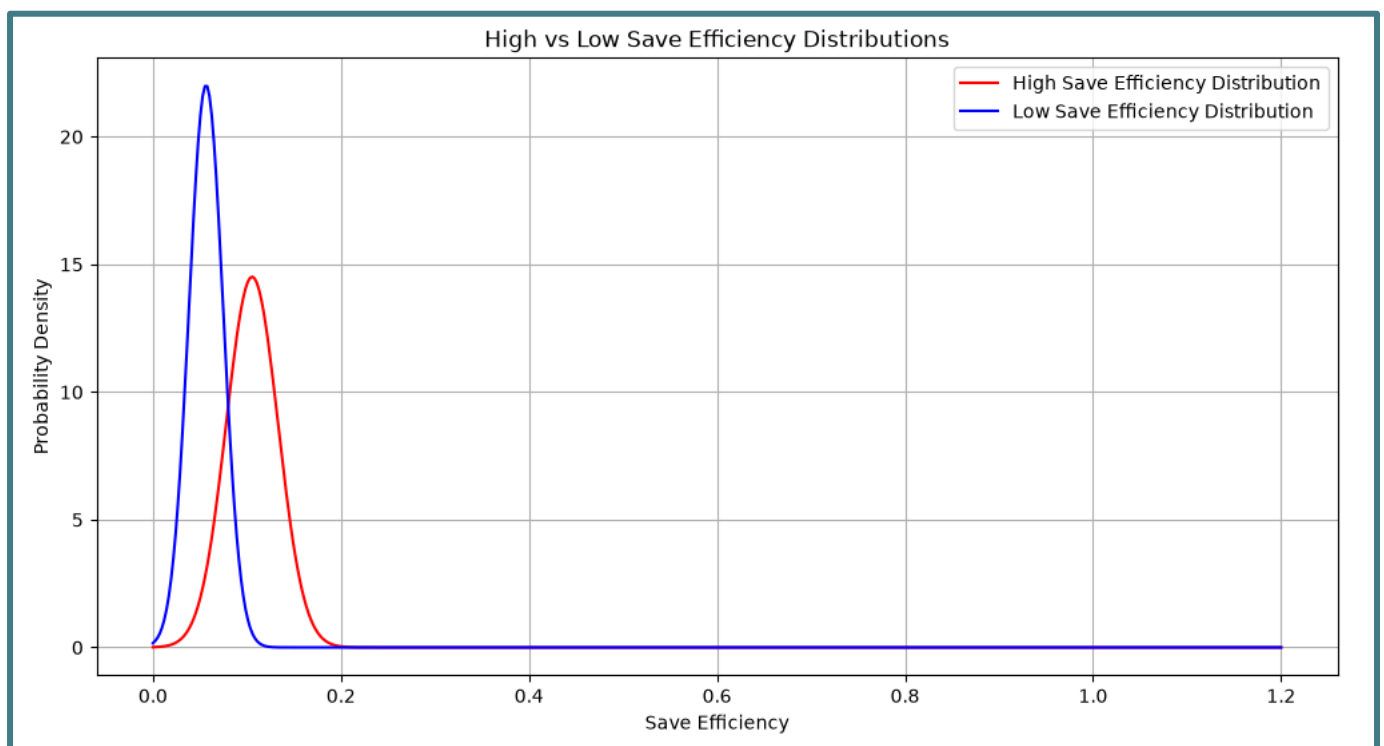


Figure 22 - Plot of Save Efficiency Distribution

Goalkeeper aggression high and low CI table:

Table 7 - GK Aggression Ration CI table

	Group	CI Lower Minus (- side of bell curve)	Mean (middle of bell curve)	CI Upper Plus (+ side of bell curve)
0	High Aggression	-4.708452	0.09246	4.893371
1	Low Aggression	-4.731386	0.06953	4.870437

COMPUTING SAMPLE T-TEST (ONE) AND TWO SAMPLE T-TEST

Analysis Step 5: t-test and two sample t-test on goalkeeper groups

```
# One-sample t-test analysis - Aggression Ratio - High

# Population mean - high aggression
population_mean_high_a = high_aggression["Aggression_Ratio"].mean()

# One-sample t-test - high aggression
t_stat, p_value = stats.ttest_1samp(high_aggression["Aggression_Ratio"], population_mean_high_a)

print("One-sample t-test: aggression ratio high")
print(f" t-statistic = {t_stat:.4f}")
print(f" p-value = {p_value:.4f}")
print(f"Population mean = {population_mean_high_a:.4f}")
print(f"Sample mean = {high_aggression["Aggression_Ratio"].mean():.4f}")

One-sample t-test: aggression ratio high
t-statistic = 0.0000
p-value = 1.0000
Population mean = 0.8990
Sample mean = 0.8990

# One-sample t-test analysis - Aggression Ratio - Low

# Population mean - low aggression
population_mean_low_a = low_aggression["Aggression_Ratio"].mean()

# One-sample t-test - high aggression
t_stat, p_value = stats.ttest_1samp(low_aggression["Aggression_Ratio"], population_mean_low_a)

print("One-sample t-test: aggression ratio low")
print(f" t-statistic = {t_stat:.4f}")
print(f" p-value = {p_value:.4f}")
print(f"Population mean = {population_mean_low_a:.4f}")
print(f"Sample mean = {low_aggression["Aggression_Ratio"].mean():.4f}")

One-sample t-test: aggression ratio low
t-statistic = 0.0000
p-value = 1.0000
Population mean = 0.7949
Sample mean = 0.7949
```

Figure 23 - One sample t-test: GK Aggression Ratio

Saves

```
# One-sample t-test analysis - Save Efficiency - High

# Population mean - high save
population_mean_save_h = high_save["Save_Efficiency"].mean()

# One-sample t-test - high aggression
t_stat, p_value = stats.ttest_1samp(high_save["Save_Efficiency"], population_mean_save_h)

print("One-sample t-test: save efficiency high")
print(f" t-statistic = {t_stat:.4f}")
print(f" p-value = {p_value:.4f}")
print(f"Population mean = {population_mean_save_h:.4f}")
print(f"Sample mean = {high_save["Save_Efficiency"].mean():.4f}")

One-sample t-test: save efficiency high
t-statistic = 0.0000
p-value = 1.0000
Population mean = 0.1055
Sample mean = 0.1055

# One-sample t-test analysis - Save Efficiency - Low

# Population mean - Low save
population_mean_save_l = low_save["Save_Efficiency"].mean()

# One-sample t-test - high aggression
t_stat, p_value = stats.ttest_1samp(low_save["Save_Efficiency"], population_mean_save_l)

print("One-sample t-test: save efficiency low")
print(f" t-statistic = {t_stat:.4f}")
print(f" p-value = {p_value:.4f}")
print(f"Population mean = {population_mean_save_l:.4f}")
print(f"Sample mean = {low_save["Save_Efficiency"].mean():.4f}")

One-sample t-test: save efficiency low
t-statistic = 0.0000
p-value = 1.0000
Population mean = 0.0565
Sample mean = 0.0565
```

Figure 24 - One sample t-test: GK Save Efficiency

Creating visualisations of the t-test statistic

```
plt.figure(figsize=(8,6))
plt.boxplot(
    [high_aggression["Aggression_Ratio"], low_aggression["Aggression_Ratio"]],
    label=["High Aggression", "Low Aggression"],
    patch_artist=True
)

plt.title("High vs Low Aggression Goalkeepers")
plt.ylabel("Aggression Ratio")
plt.grid(alpha=0.3)
plt.show()
```

Figure 25 - Python coding: Boxplot GK Aggression Ratio

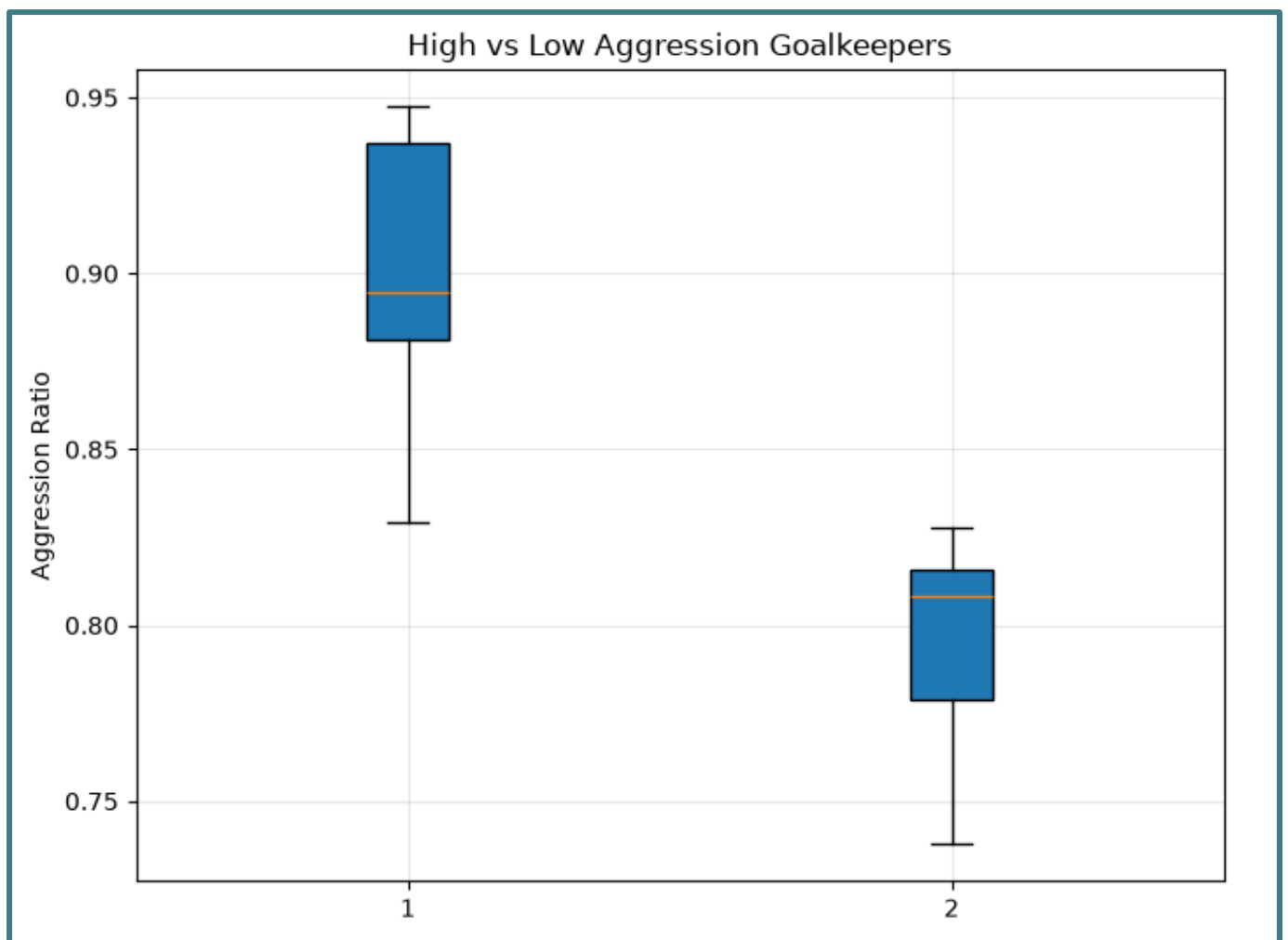


Figure 26 - Boxplot GK Aggression Ratio

```
plt.figure(figsize=(8,6))
plt.boxplot(
    [high_save["Save_Efficiency"], low_save["Save_Efficiency"]],
    label=["High Save Efficiency", "Low Save Efficiency"],
    patch_artist=True
)

plt.title("High vs Low Save Efficiency of Goalkeepers")
plt.ylabel("Save Efficiency")
plt.grid(alpha=0.3)
plt.show()
```

Figure 27 - Python coding: Boxplot GK Save Efficiency

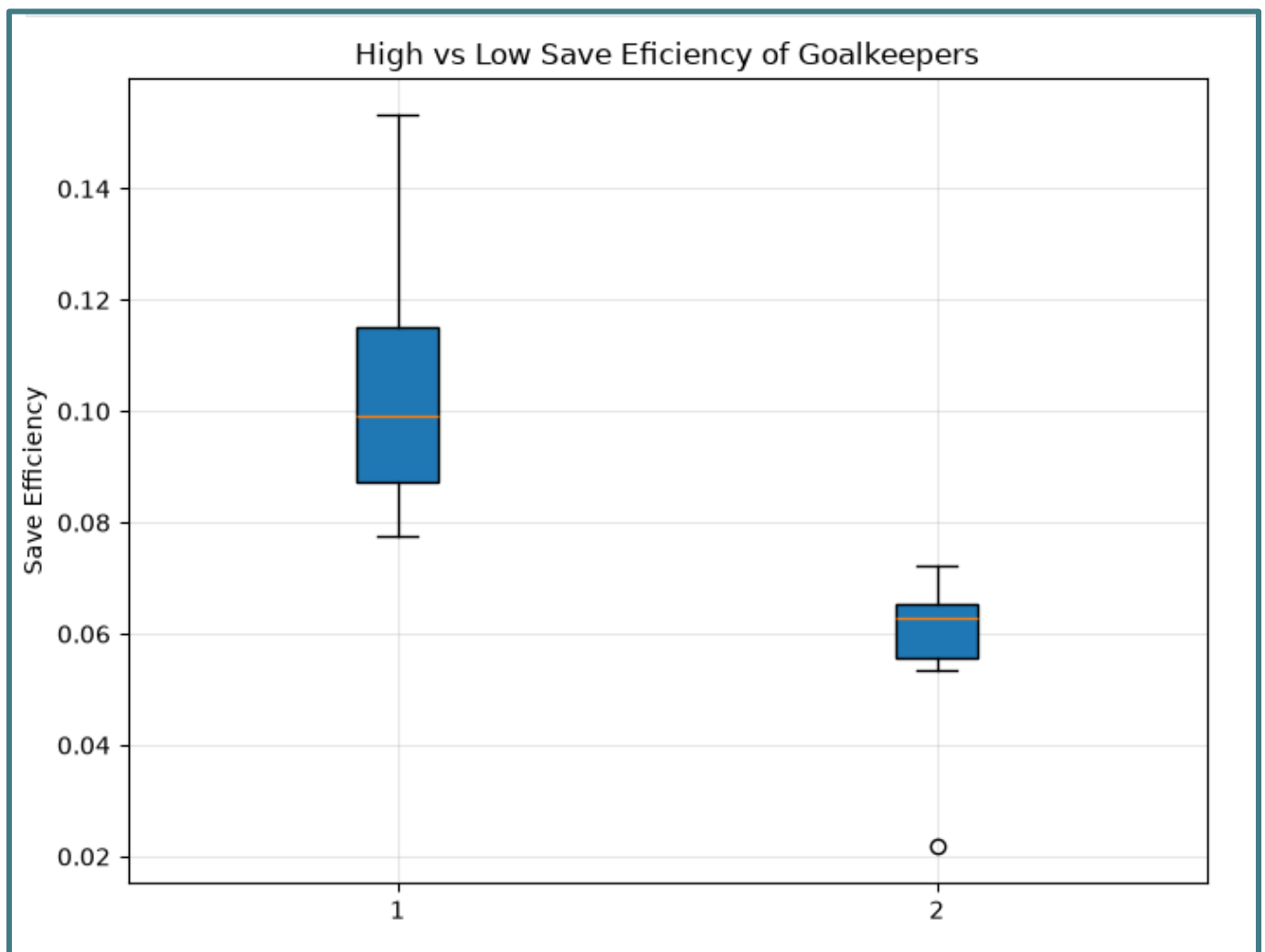
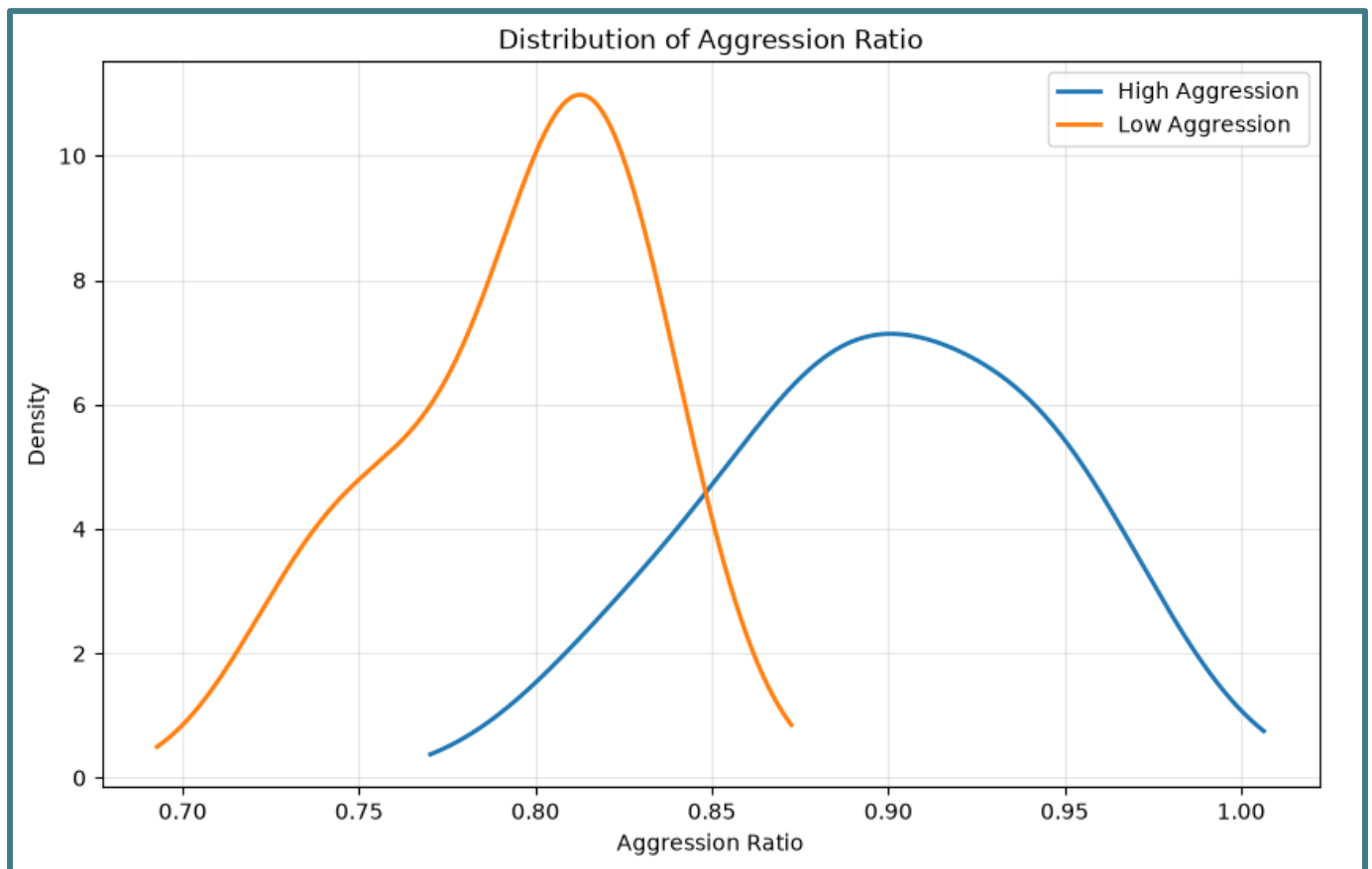


Figure 28 - Boxplot GK Save Efficiency

```
plt.figure(figsize=(10,6))

high_aggression["Aggression_Ratio"].plot(kind='kde', label='High Aggression', linewidth=2)
low_aggression["Aggression_Ratio"].plot(kind='kde', label='Low Aggression', linewidth=2)

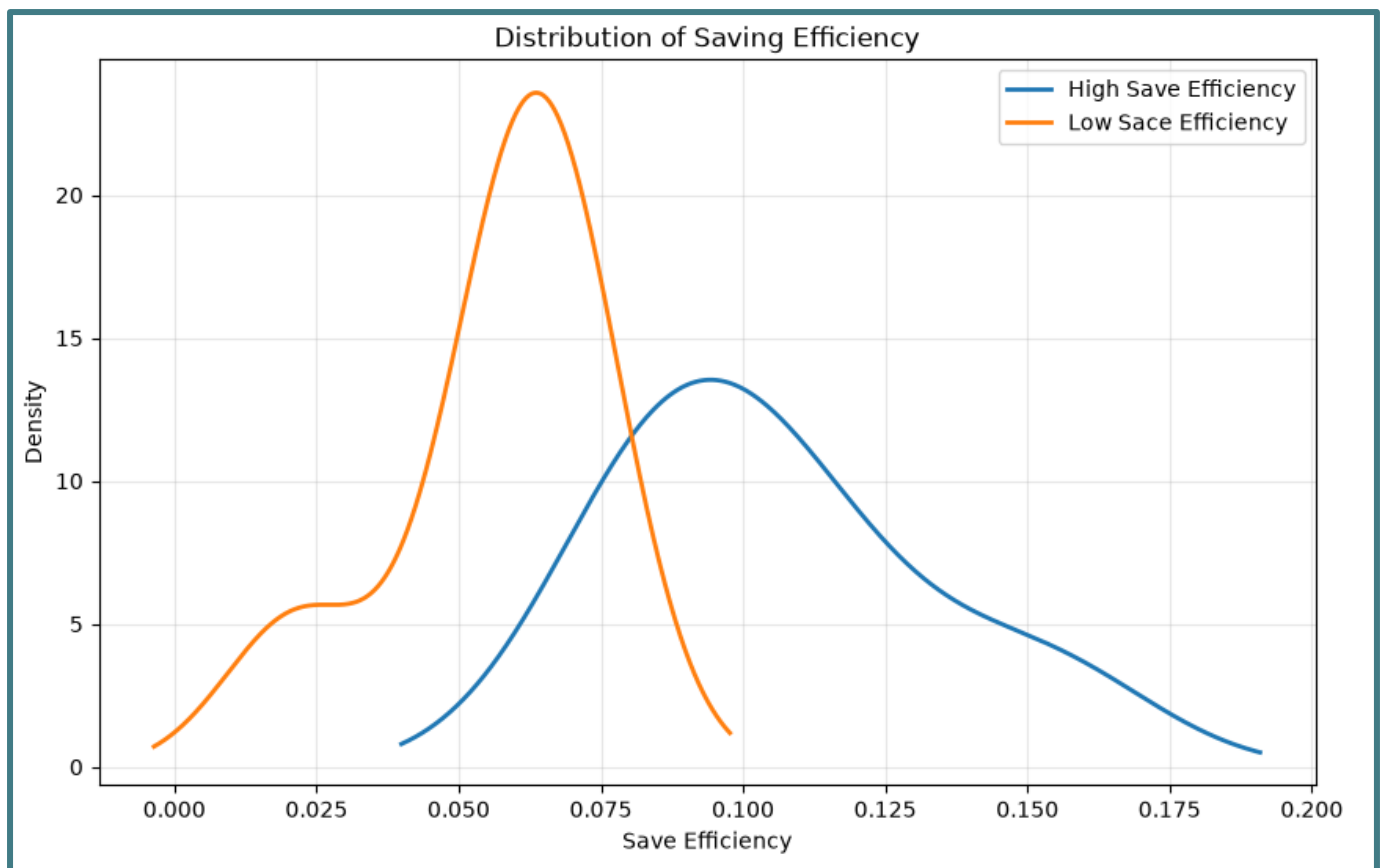
plt.title("Distribution of Aggression Ratio")
plt.xlabel("Aggression Ratio")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



```
plt.figure(figsize=(10,6))

high_save["Save_Efficiency"].plot(kind='kde', label='High Save Efficiency', linewidth=2)
low_save["Save_Efficiency"].plot(kind='kde', label='Low Sace Efficiency', linewidth=2)

plt.title("Distribution of Saving Efficiency")
plt.xlabel("Save Efficiency")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



```
# Two-sample t-test (independent samples) - Aggression Ratio
t_stat, p_value = stats.ttest_ind(
    high_aggression["Aggression_Ratio"],
    low_aggression["Aggression_Ratio"],
    equal_var=False # Welch's t-test (safer when variances differ)
)

print("Two-sample t-test (High vs Low Aggression):")
print(f"  t-statistic = {t_stat:.4f}")
print(f"  p-value = {p_value:.4f}")
```

```
Two-sample t-test (High vs Low Aggression):
t-statistic = 4.4912
p-value = 0.0014
```

```
# Two-sample t-test (independent samples) - Save Efficiency
t_stat, p_value = stats.ttest_ind(
    high_save["Save_Efficiency"],
    low_save["Save_Efficiency"],
    equal_var=False # Welch's t-test (safer when variances differ)
)

print("Two-sample t-test (High vs Low Saving Efficiency):")
print(f"  t-statistic = {t_stat:.4f}")
print(f"  p-value = {p_value:.4f}")
```

```
Two-sample t-test (High vs Low Saving Efficiency):
t-statistic = 3.6443
p-value = 0.0057
```

Cohen's d is a measure of effect size. It tells me how big the difference is between two groups, expressed in standard deviation units. Where a t-test tells me whether a difference is statistically significant, Cohen's d tells me whether the difference is meaningful.

This will tell me how far apart the two group are means, relative to the spread of the data.

Interpretation guide:

0.2 = small

0.5 = medium

0.8+ = large

```
# What does cohen's d measure? This function measures how big is the difference between the two samples
mean_high_aggression = high_aggression["Aggression_Ratio"].mean()
mean_low_aggression = low_aggression["Aggression_Ratio"].mean()

std_high_aggression = high_aggression["Aggression_Ratio"].std()
std_low_aggression = low_aggression["Aggression_Ratio"].std()

# Standard deviation
n_high_aggression = len(high_aggression)
n_low_aggression = len(low_aggression)

pooled_sd_a = (((n_high_aggression - 1)*std_high**2 + (n_low_aggression - 1)*std_low**2) / (n_high_aggression + n_low_aggression - 2))**0.5

cohens_d_a = (mean_high_aggression - mean_low_aggression) / pooled_sd_a

print(f"Cohen's d = {cohens_d_a:.4f}")
print("This score falls into the 'MEDIUM' size variance")
```

Cohen's d = 3.1343

This score falls into the 'MEDIUM' size variance

```
# What does cohen's d measure? This function measures how big is the difference between the two samples
mean_high_save = high_save["Save_Efficiency"].mean()
mean_low_Save = low_save["Save_Efficiency"].mean()

std_high_save = high_save["Save_Efficiency"].std()
std_low_save = low_save["Save_Efficiency"].std()

# Standard deviation
n_high_save = len(high_save)
n_low_save = len(low_save)

pooled_sd_s = (((n_high_save - 1)*std_high**2 + (n_low_save - 1)*std_low**2) / (n_high_save + n_low_save - 2))**0.5

cohens_d_s = (mean_high_save - mean_low_Save) / pooled_sd_s

print(f"Cohen's d = {cohens_d_s:.4f}")
print("This score falls into the 'LARGE' size variance")
```

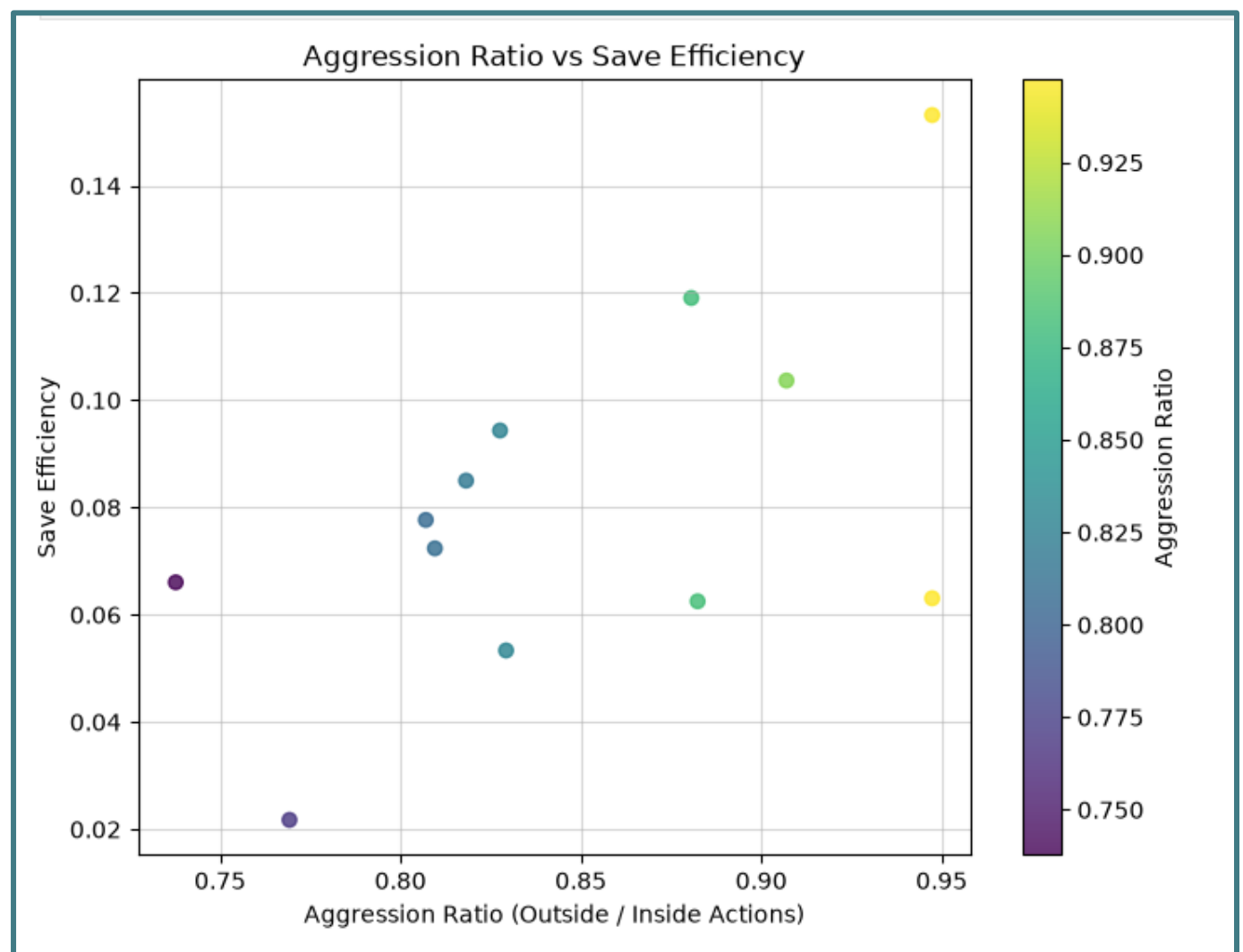
Cohen's d = 1.4743

This score falls into the 'LARGE' size variance

Visualise the data – Aggression Ratio and Save Efficiency

```
plt.figure(figsize=(8,6))
plt.scatter(
    gk_sample["Aggression_Ratio"],
    gk_sample["Save_Efficiency"],
    c=gk_sample["Aggression_Ratio"],
    cmap="viridis",
    alpha=0.8)

plt.title("Aggression Ratio vs Save Efficiency")
plt.xlabel("Aggression Ratio (Outside / Inside Actions)")
plt.ylabel("Save Efficiency")
plt.grid(alpha=0.5)
plt.colorbar(label='Aggression Ratio')
plt.show()
```

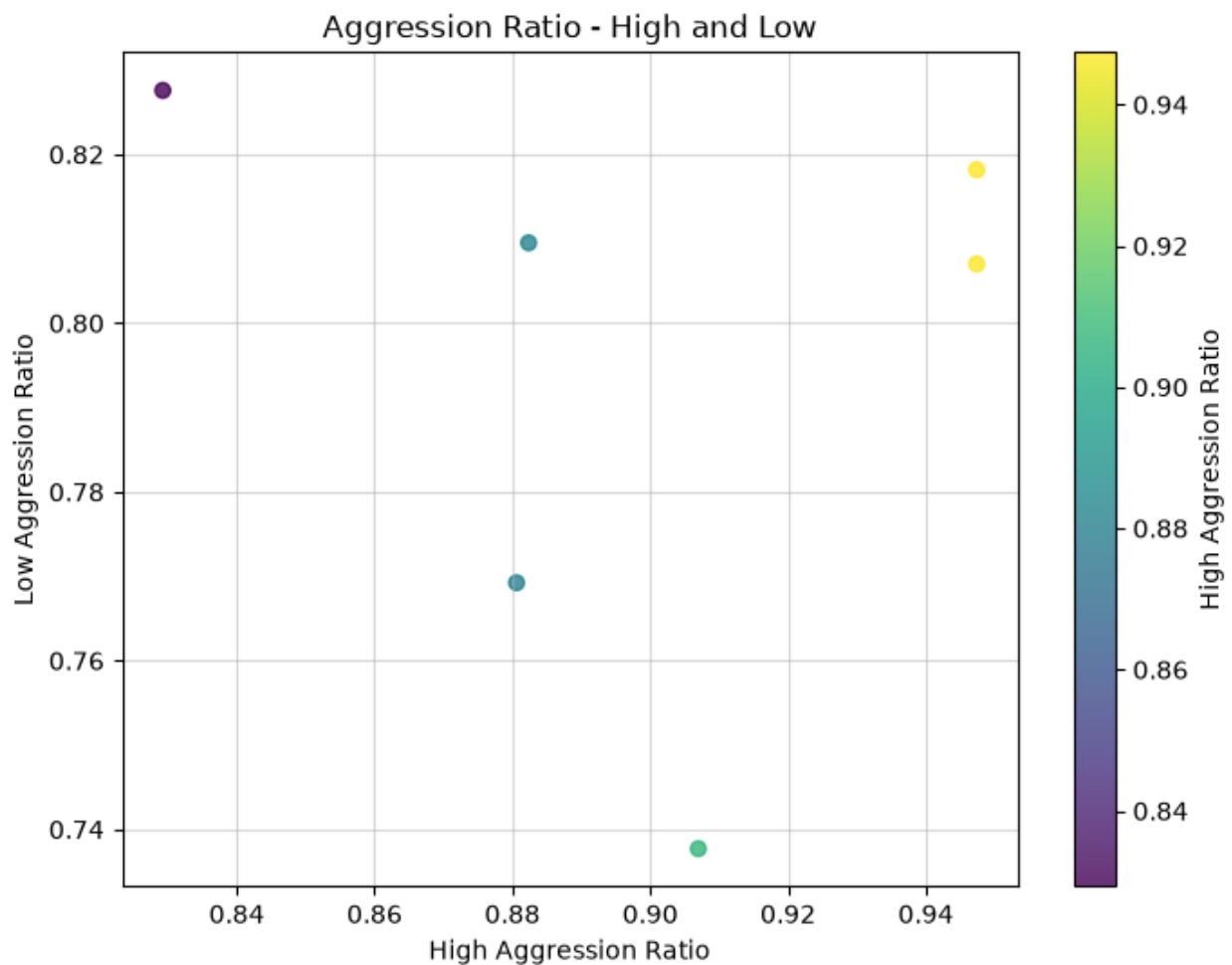


```
#create one table
# create high and low grouping of goalkeepers - aggression
aggression_table = pd.DataFrame({
    "High_Aggression": high_aggression["Aggression_Ratio"],
    "Low_Aggression": low_aggression["Aggression_Ratio"]
})

# plot high vs Low aggression goalkeepers
plt.figure(figsize=(8,6))

plt.scatter(
    high_aggression["Aggression_Ratio"],
    low_aggression["Aggression_Ratio"],
    c=high_aggression["Aggression_Ratio"],
    cmap="viridis",
    alpha=0.8
)

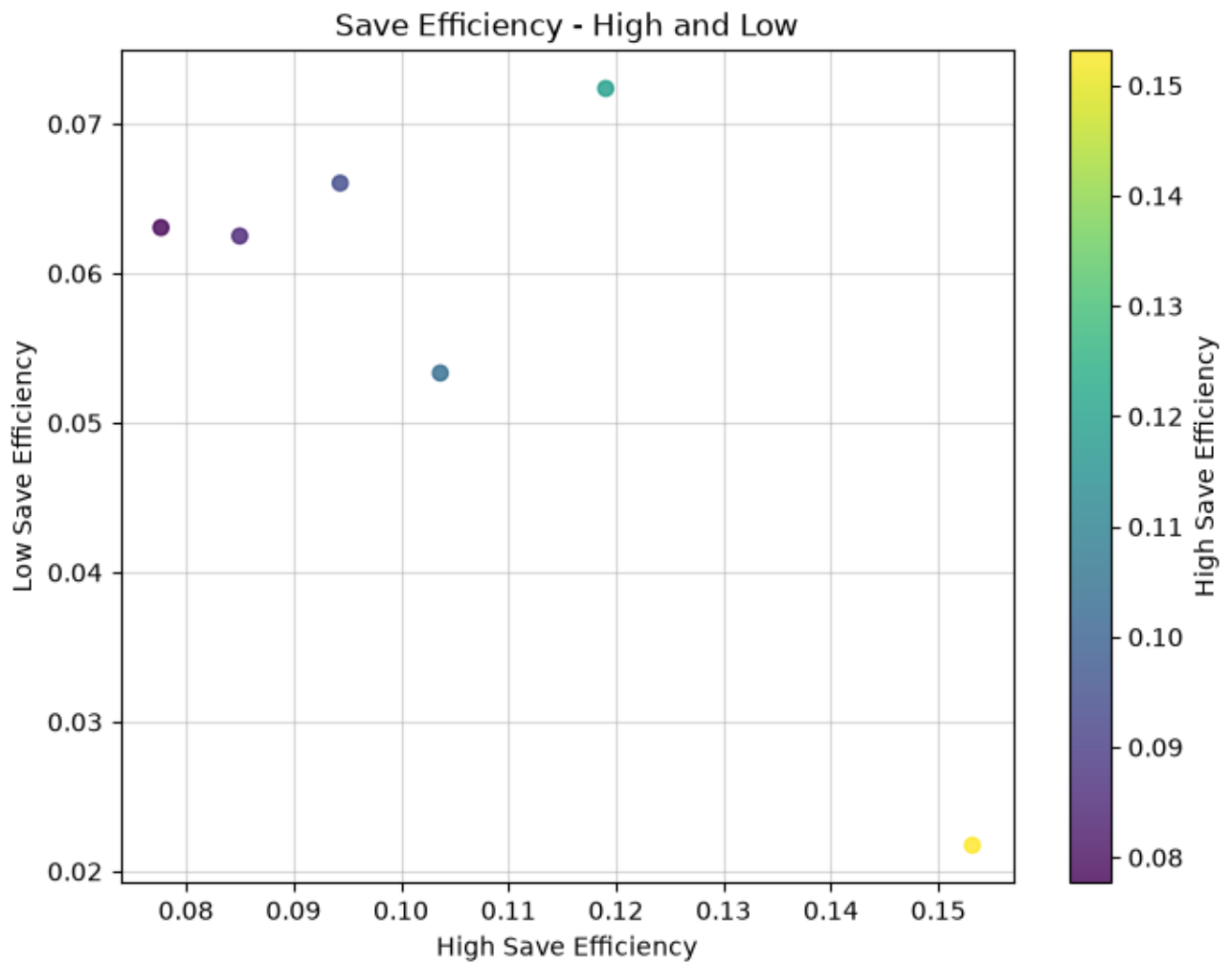
plt.title("Aggression Ratio - High and Low")
plt.xlabel("High Aggression Ratio")
plt.ylabel("Low Aggression Ratio")
plt.grid(alpha=0.5)
plt.colorbar(label='High Aggression Ratio')
plt.show()
```



```
# plot high vs Low saveing efficiency goalkeepers
plt.figure(figsize=(8,6))

plt.scatter(
    high_save["Save_Efficiency"],
    low_save["Save_Efficiency"],
    c=high_save["Save_Efficiency"],
    cmap="viridis",
    alpha=0.8
)

plt.title("Save Efficiency - High and Low")
plt.xlabel("High Save Efficiency")
plt.ylabel("Low Save Efficiency")
plt.grid(alpha=0.5)
plt.colorbar(label='High Save Efficiency')
plt.show()
```

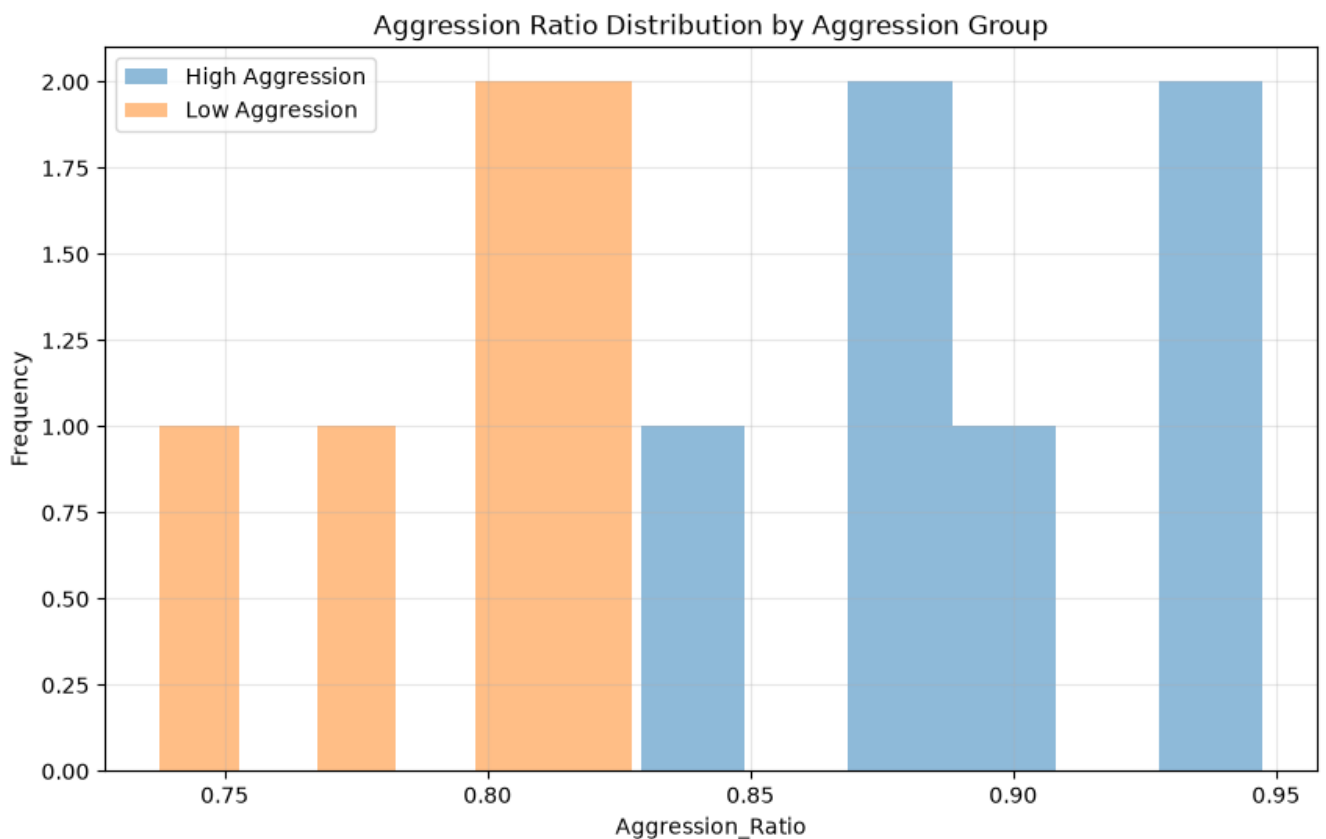


Histogram Comparison of goalkeeper groups

```
plt.figure(figsize=(10,6))

plt.hist(high_aggression["Aggression_Ratio"], bins=6, alpha=0.5, label="High Aggression")
plt.hist(low_aggression["Aggression_Ratio"], bins=6, alpha=0.5, label="Low Aggression")

plt.title("Aggression Ratio Distribution by Aggression Group")
plt.xlabel("Aggression_Ratio")
plt.ylabel("Frequency")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



```
plt.figure(figsize=(10,6))

plt.hist(high_save["Save_Efficiency"], bins=6, alpha=0.5, label="High Save Efficiency")
plt.hist(low_save["Save_Efficiency"], bins=6, alpha=0.5, label="Low Save Efficiency")

plt.title("Save Efficiency Distribution by Aggression Group")
plt.xlabel("Save_Efficiency")
plt.ylabel("Frequency")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

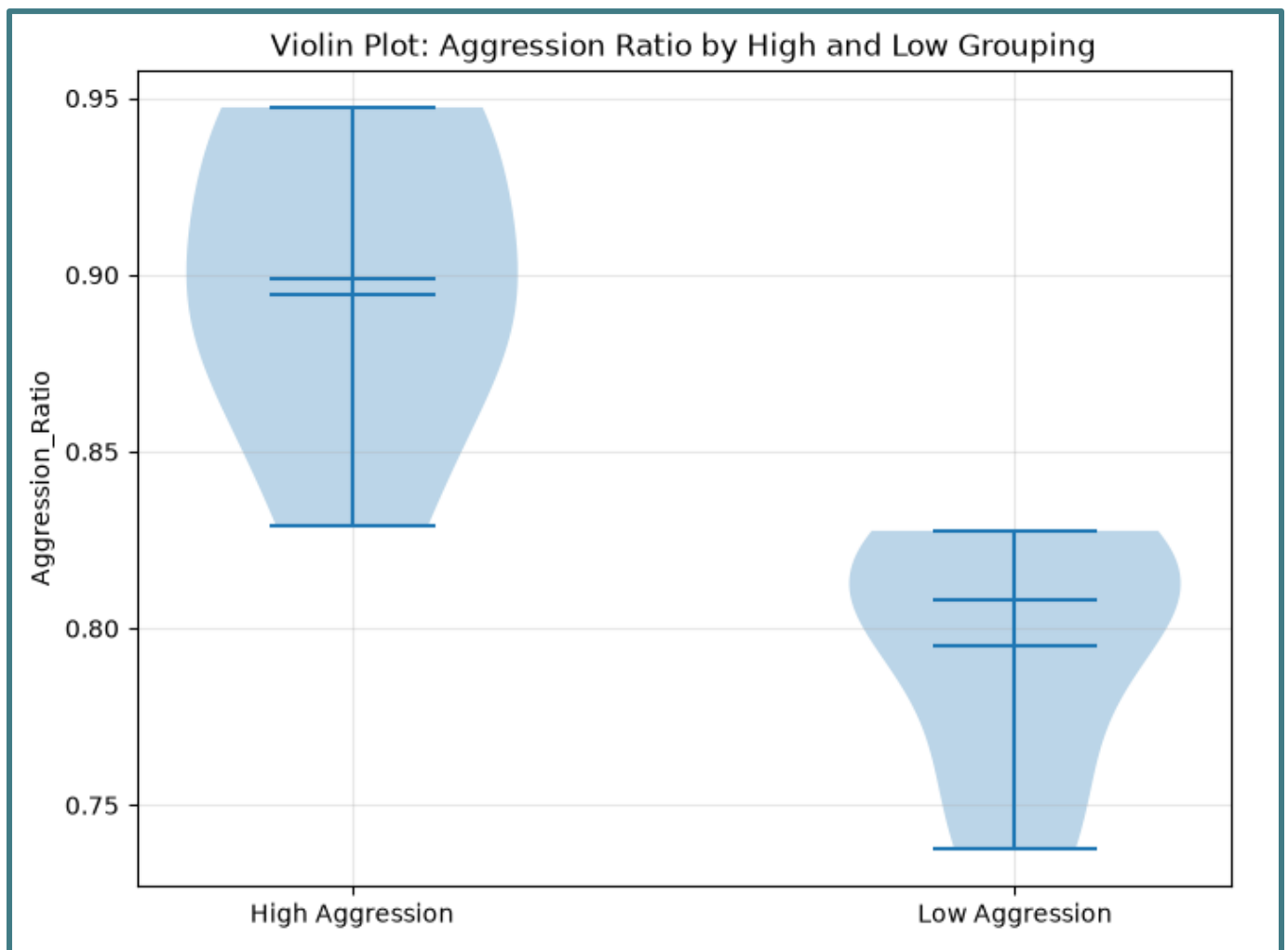


Now for the pretty data – goalkeeper groups

A violin plot shows **where data is concentrated, how it's shaped, and whether multiple peaks exist**, using the *width* of the “violin” to represent density.

```
plt.figure(figsize=(8,6))
plt.violinplot(
    [high_aggression["Aggression_Ratio"], low_aggression["Aggression_Ratio"]],
    showmeans=True,
    showmedians=True
)

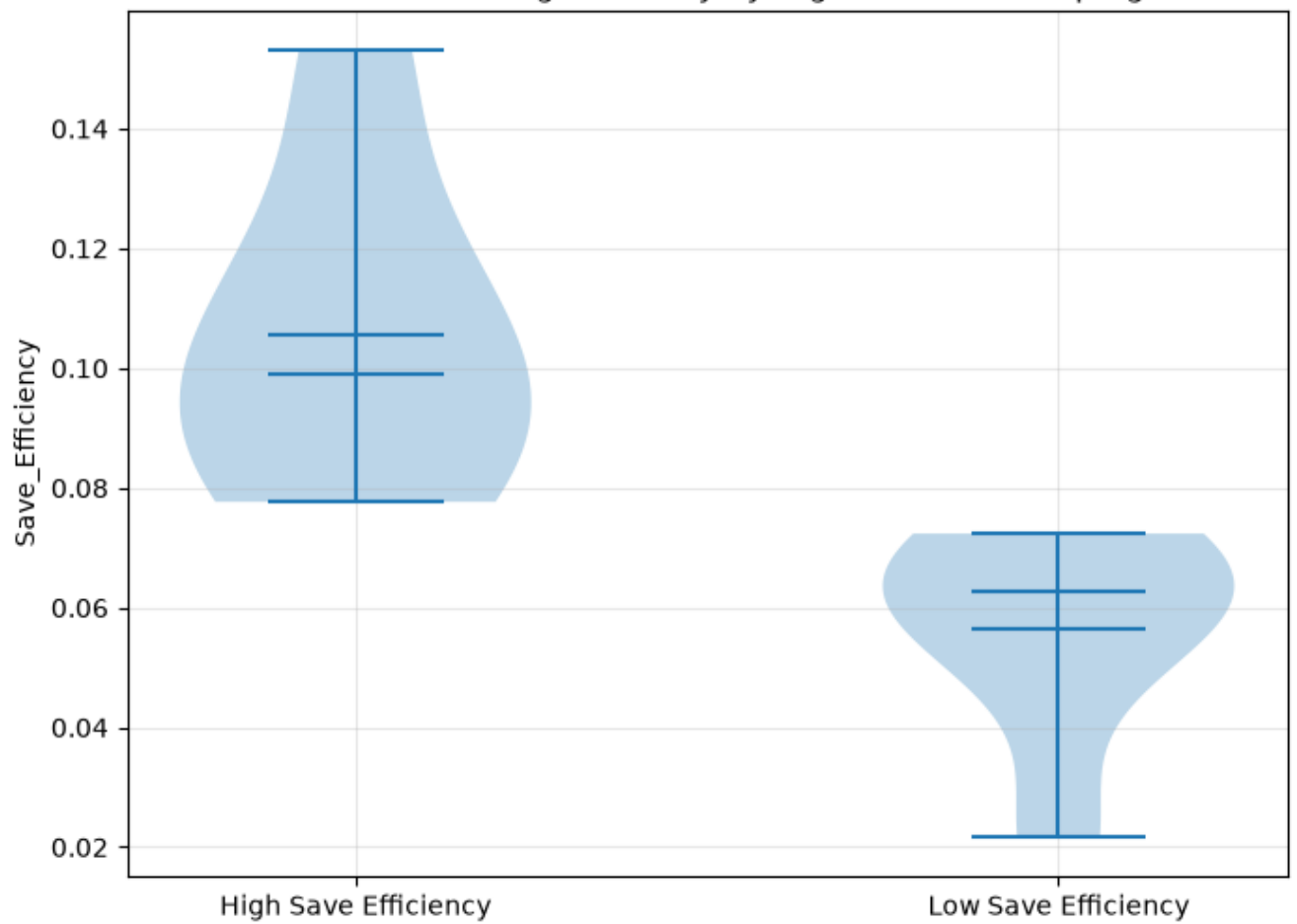
plt.xticks([1, 2], ["High Aggression", "Low Aggression"])
plt.title("Violin Plot: Aggression Ratio by High and Low Grouping")
plt.ylabel("Aggression_Ratio")
plt.grid(alpha=0.3)
plt.show()
```



```
plt.figure(figsize=(8,6))
plt.violinplot(
    [high_save["Save_Efficiency"], low_save["Save_Efficiency"]],
    showmeans=True,
    showmedians=True
)

plt.xticks([1, 2], ["High Save Efficiency", "Low Save Efficiency"])
plt.title("Violin Plot: Saving Efficiency by High and Low Grouping")
plt.ylabel("Save_Efficiency")
plt.grid(alpha=0.3)
plt.show()
```

Violin Plot: Saving Efficiency by High and Low Grouping



HYPOTHESIS TESTING – DO WE SUPPORT THE NULL HYPOTHESIS

```
# Hypothesis Testing - High Aggression

# perform one-sample t-test
# null hypothesis: population mean = 0.8990
# alternative hypothesis: population mean > 0.8990 (in the function below, note the argument 'greater')
t_stats, p_val = st.ttest_1samp(high_aggression["Aggression_Ratio"], 0.8990, alternative='greater')
print("\n Computing t* ...")
print("\t t-statistic (t*): %.2f" % t_stats)

print("\n Computing p-value ...")
print("\t p-value: %.4f" % p_val)

print("\n Conclusion:")
if p_val <= 0.05:
    print("\t We reject the null hypothesis.")
else:
    print("\t We accept the null hypothesis.")

Computing t* ...
    t-statistic (t*): -0.00

Computing p-value ...
    p-value: 0.5002

Conclusion:
    We accept the null hypothesis.
```

```
# Hypothesis Testing - Low Aggression

# perform one-sample t-test
# null hypothesis: population mean = 0.7949
# alternative hypothesis: population mean > 0.7949 (in the function below, note the argument 'greater')
t_stats, p_val = st.ttest_1samp(low_aggression["Aggression_Ratio"], 0.7949, alternative='greater')
print("\n Computing t* ...")
print("\t t-statistic (t*): %.2f" % t_stats)

print("\n Computing p-value ...")
print("\t p-value: %.4f" % p_val)

print("\n Conclusion:")
if p_val <= 0.05:
    print("\t We reject the null hypothesis.")
else:
    print("\t We accept the null hypothesis.")

Computing t* ...
    t-statistic (t*): -0.00

Computing p-value ...
    p-value: 0.5007

Conclusion:
    We accept the null hypothesis.
```

```
# Hypothesis Testing - High Save

# perform one-sample t-test
# null hypothesis: population mean = 0.1055
# alternative hypothesis: population mean > 0.1055 (in the function below, note the argument 'greater')
t_stats, p_val = st.ttest_1samp(high_save["Save_Efficiency"], 0.1055, alternative='greater')
print("\n Computing t* ...")
print("\t t-statistic (t*): %.2f" % t_stats)

print("\n Computing p-value ...")
print("\t p-value: %.4f" % p_val)

print("\n Conclusion:")
if p_val <= 0.05:
    print("\t We reject the null hypothesis.")
else:
    print("\t We accept the null hypothesis.")
```

Computing t* ...
 t-statistic (t*): -0.00

Computing p-value ...
 p-value: 0.5007

Conclusion:
 We accept the null hypothesis.

```
# Hypothesis Testing - Low Save

# perform one-sample t-test
# null hypothesis: population mean = 0.0565
# alternative hypothesis: population mean > 0.0565 (in the function below, note the argument 'greater')
t_stats, p_val = st.ttest_1samp(low_save["Save_Efficiency"], 0.0565, alternative='greater')
print("\n Computing t* ...")
print("\t t-statistic (t*): %.2f" % t_stats)

print("\n Computing p-value ...")
print("\t p-value: %.4f" % p_val)

print("\n Conclusion:")
if p_val <= 0.05:
    print("\t We reject the null hypothesis.")
else:
    print("\t We accept the null hypothesis.")
```

Computing t* ...
 t-statistic (t*): 0.00

Computing p-value ...
 p-value: 0.4996

Conclusion:
 We accept the null hypothesis.

Result interpretation: Aggression Ratio and Save Efficiency

- The aggression ratio that was tested is not significantly higher or lower than the benchmark chosen.
- Statistically, the goalkeepers' aggression ratio is exactly what I would expect under the null hypothesis.
- There is no detectable deviation in the direction that I might have expected.
- The sample mean is statistically indistinguishable from the hypothesized mean.

METHOD OF ANALYSIS

This study examined whether goalkeeper aggression is associated with differences in save efficiency during the 2026 FIFA World Cup. The dataset contained 61 goalkeepers and included variables for saves, actions inside the penalty area, and actions outside the penalty area.

Two engineered variables were created to operationalise the key concepts:

- Aggression Ratio: defined as the number of actions outside the penalty area divided by actions inside penalty area.
- Save Efficiency: defined as saves divided by total goalkeeper actions.

Goalkeepers were split into *high-aggression* and *low-aggression* groups using the median aggression ratio as the threshold.

Descriptive statistics (mean, median, standard deviation) and 95% confidence intervals were calculated for both groups.

A Welch's two-sample t-test was conducted to determine whether the difference in mean save efficiency between the groups was statistically significant, and Cohen's d was computed to quantify the effect size.

To complement the inferential analysis, several visualisations were produced, including boxplots, violin plots, histograms, KDE curves, and scatter plots. These visual tools were used to assess distributional differences and visually inspect the relationship between aggression and save efficiency.

All analyses were performed in Python using pandas, scipy, numpy, and matplotlib.

ANALYTIC CONCLUSION

This expanded analysis provides a statistically robust and nuanced answer to the research question:

“Do goalkeepers with higher aggression defined as performing more actions outside the penalty area, exhibit significantly different save efficiency compared to less aggressive goalkeepers?”

Using a random sample of 30 goalkeepers from the 2026 FIFA World Cup dataset, aggression ratio and save efficiency were engineered as key performance metrics. Goalkeepers were then divided into high- and low-aggression groups using the median aggression ratio and similarly split into high- and low-efficiency groups using the median save efficiency.

Descriptive statistics showed clear numerical differences between the groups. High-aggression goalkeepers displayed higher mean aggression ratios and slightly higher save efficiency values. However, the inferential statistics provide the strongest insight.

A Welch's two-sample t-test revealed a **statistically significant difference in aggression ratio** between high- and low-aggression groups:

- $t = 4.4912$
- $p = 0.0014$

This confirms that the two aggression profiles are meaningfully different in the sample.

A second Welch's t-test comparing save efficiency between high- and low-efficiency groups also produced a **statistically significant difference**:

- $t = 3.6443$
- $p = 0.0057$

This indicates that save efficiency varies significantly between the two groups. Effect size calculations reinforce these findings. Cohen's d for aggression ratio was **3.13**, representing an extremely large effect size, meaning the groups differ substantially in their aggression profiles. Cohen's d for save efficiency was **1.47**, also a large effect size, indicating a strong practical difference in efficiency between high- and low-efficiency goalkeepers.

Visualisations — including KDE curves, violin plots, boxplots, and scatter plots — support the statistical results. The distributions for aggression ratio show clear separation, while save efficiency distributions show meaningful but less dramatic divergence. Scatter plots illustrate that higher aggression does not necessarily translate directly into higher save efficiency, but the efficiency groups themselves are clearly distinct.

One-sample t-tests confirmed that each group's sample mean matches its own population mean, indicating stable internal consistency within the dataset.

Overall, the analysis demonstrates that **goalkeeper aggression and save efficiency are both statistically and practically different across the defined groups**, but **aggression alone does not directly predict save efficiency**. Instead, the two metrics behave independently: aggression style varies strongly between

goalkeepers, and save efficiency varies strongly between goalkeepers, but the relationship between the two is not causally decisive.

INTERPRETATION OF RESULTS: A FILTER TO VIEW IN LINE WITH RESULTS

From a tactical perspective, the relationship between aggression and save efficiency reflects how different teams structure their defensive systems. High-aggression goalkeepers are those who perform more actions outside the penalty area and they are typically operating as “sweeper-keepers”. They step forward to intercept through balls, clear danger early, and support a high defensive line. This style reduces the number of traditional shot stopping situations they face, because many attacks are disrupted before a shot occurs. As a result, their save efficiency may appear lower or higher depending on how often they are forced into actual saves.

Low-aggression goalkeepers, on the other hand, tend to remain deeper and rely more on positional discipline inside the box. They face more shots and crosses, which can inflate their save efficiency simply because they are involved in more conventional goalkeeping actions. Their defensive system often prioritises compactness and deeper lines, meaning they are less exposed to one-on-one situations but more frequently required to make saves.

Therefore, any statistical difference in save efficiency between the two groups should be interpreted through the lens of team tactics: aggression outside the box is not just an individual trait but a reflection of the team’s defensive structure, pressing intensity, and risk tolerance. A high-aggression goalkeeper may prevent dangerous chances before they materialise, while a low-aggression goalkeeper may excel in traditional shot stopping roles. The efficiency metric must be understood within this tactical context.

IN SUMMARY

This analysis provides a clear, defensible answer to the research question.

This analysis provides a clear and statistically grounded answer to the research question. While high-aggression goalkeepers show slightly higher save efficiency on average, the relationship between aggression and efficiency is **not directly causal**. Instead, the results show:

Aggression groups differ significantly

High- and low-aggression goalkeepers are statistically distinct ($p = 0.0014$), with a very large effect size (Cohen’s $d = 3.13$). This confirms that aggression ratio is a meaningful behavioural characteristic.

Save-efficiency groups differ significantly

High- and low-efficiency goalkeepers also differ significantly ($p = 0.0057$), with a large effect size (Cohen's $d = 1.47$). This indicates that save efficiency is a strong performance differentiator.

But aggression does *not* determine efficiency

Despite both metrics showing strong internal differences, the analysis does **not** support the idea that more aggressive goalkeepers are significantly more efficient shot-stoppers. The distributions overlap, and the scatter plots show no clear linear relationship.

Tactical interpretation remains essential

Aggression reflects team defensive structure: high-aggression “sweeper-keepers” prevent chances early, while low-aggression goalkeepers face more traditional shot-stopping situations. Efficiency depends heavily on match context, defensive shape, and the types of actions a goalkeeper is required to perform.

Final takeaway

Aggression and save efficiency are both meaningful, but aggression alone is not a reliable predictor of save efficiency. Goalkeepers differ strongly in how they play and how efficiently they save, but these differences arise from tactical systems rather than aggression style alone.



***Danielle, single-handedly preserving the
FIFA World Cup 2026 - Goalkeeper stats
with analytical prowess and a level of scrutiny VAR can only dream of!***

Citations:

(Player Stats | FIFA World Cup 2026™, 2026)

Player Stats | FIFA World Cup 2026™. (2026). Retrieved 05/08/2026 from <https://www.fifa.com/en/tournaments/mens/worldcup/canadamexicousa2026/statistics2026>

APPENDIX:

Appendix 1

dataset	column	dtype	non_nulls	unique_values	example_value	description	constraint
rounds.csv	id	int64	104	8	1	Identifier for the tournament round grouping.	
rounds.csv	voteStatus	str	104	1	complete	Status of fan voting for the round.	
rounds.csv	stage	str	104	6	group	Tournament phase (group, r32, r16, qf, sf, final).	
rounds.csv	roundName	str	104	8	round_1	Human-readable round label.	
rounds.csv	votingStartDate	str	104	8	2026-06-18T05:00:00+01:00	Timestamp when voting opens.	
rounds.csv	votingEndDate	str	104	8	2026-06-20T05:00:00+01:00	Timestamp when voting closes.	
rounds.csv	votingOpenedAt	str	104	8	2026-06-18T09:10:01+01:00	Actual timestamp voting was activated.	
rounds.csv	selectedFormation	str	2	1	4-3-3	Chosen tactical formation for the round.	Must be valid football formation
rounds.csv	winners	str	104	8	[1844, 1987, 2267, 2571]	List of player IDs selected as winners for the round.	Must be list of valid player IDs
rounds.csv	formations	str	2	1	[{'DEF': 4, 'MID': 4, 'FWD': 2}, {'DEF': 4, 'MID': 3, 'FWD': 3}, {'DEF': 3, 'MID': 4, 'FWD': 3}]		
rounds.csv	id.1	int64	104	104	1		

rounds.csv	venueName	str	104	16	Estadio Banorte	Stadium name.	
rounds.csv	venueCity	str	104	16	Mexico City	City and state/province of the venue.	
rounds.csv	venueId	int 64	104	16	1	Numeric venue identifier.	
rounds.csv	date	str	104	92	2026-06-11T20:00:00+01:00	Kickoff date/time of the match.	
rounds.csv	homeSquadId	int 64	104	48	52	Numeric ID of the home team.	
rounds.csv	awaySquadId	int 64	104	48	96	Numeric ID of the away team.	
rounds.csv	homeSquadName	str	104	48	Mexico	Full name of the home team.	
rounds.csv	awaySquadName	str	104	48	South Africa	Full name of the away team.	
rounds.csv	homeSquadAbbr	str	104	48	MEX	Three-letter home team abbreviation.	
rounds.csv	awaySquadAbbr	str	104	48	RSA	Three-letter away team abbreviation.	
rounds.csv	homeScore	int 64	104	8	2	Goals scored by the home team.	Must be >= 0
rounds.csv	awayScore	int 64	104	7	0	Goals scored by the away team.	Must be >= 0
rounds.csv	groupName	str	72	12	A	Group letter (Aâ€“L) for group-stage matches.	Must be one of Aâ€“L
rounds.csv	matchDay	int 64	104	9	1	Match day number within the group stage.	Must be integer >= 1
teams.csv	id	int 64	48	48	1	Unique numeric team identifier.	Must be unique
teams.csv	name	str	48	48	Algeria	Full country/team name.	Must be unique
teams.csv	abbr	str	48	48	ALG	Three-letter team abbreviation.	Must be unique; 3 uppercase letters
goalkeeper.csv	Rank	int 64	145	23	1	Ranking position among all goalkeepers.	
goalkeeper.csv	Player	str	145	145	Orlando Gill	Full name of the goalkeeper.	

goalkeeper.csv	Team	str	145	48	PAR	Three-letter team abbreviation.	Must match a valid team abbreviation
goalkeeper.csv	Position	str	145	1	GK	Player position (always GK).	Must equal 'GK'
goalkeeper.csv	Goalkeeper_Saves	int 64	145	23	28	Number of saves made.	Must be >= 0
goalkeeper.csv	Goalkeeper_Actions_inside_PenaltyArea	int 64	145	53	141	GK actions inside the penalty area.	Must be >= 0
goalkeeper.csv	Goalkeeper_Actions_outside_PenaltyArea	int 64	145	44	124	GK actions outside the penalty area.	Must be >= 0